

Design of a Kolmogorov-Arnold Network Hardware Accelerator

Fuad Mammadzada
`fuad.mammadzada.6315@student.lu.se`

Department of Electrical and Information Technology
Lund University

Supervisors: Dr. Joachim Rodrigues, Masoud Nouripayam,
Kristoffer Westring

Examiner: Dr. Pietro Andreani

June 18, 2025

Acknowledgement

I would like to thank Dr. Joachim Rodrigues for his supervision and for welcoming me into the research environment at EIT. His insights and experience helped shape the direction of this work, and I appreciated his calm and focused approach throughout the project. I am also especially grateful to my co-supervisors, Masoud Nouripayam and Kristoffer Westring — not only for introducing me to this project, but for trusting me to take it forward. This project would not have happened without their initiative, and I appreciate the freedom and responsibility they gave me throughout the process. A special thanks to Kristoffer as well, for all the pistachios he generously shared during the last semester.

I am most indebted to my family for their unwavering support and encouragement. To my father, who sparked my passion for science and inspired me to become an engineer. To my mother, whose care and countless sacrifices made everything possible. I hope this work stands as a small testament to the value of those sacrifices. To my little sisters, whose support and presence, whether near or far, have been a constant source of motivation.

I am deeply grateful to my dearest Nazrin, whose unwavering belief in me never faded, even when I doubted myself. Her love, patience, and quiet strength have been the anchor that steadied me through the most challenging parts of this journey, making it all the more meaningful. Though miles apart, her presence has never felt distant — she has remained my constant source of joy, strength, and comfort, reminding me what truly matters.

I am also incredibly thankful to the people I worked with during the past few months in the department. William's positive and friendly attitude, along with our mix of insightful and completely random conversations, made this journey a lot more enjoyable and worthwhile. I'm also grateful for the discussions with Alex, Sergio, and Arturo. Each of you had a meaningful influence on the direction of this work, and I truly appreciate it.

Gratitude goes to my friend Aryan, whose early collaboration and shared dedication helped shape the foundation of this project. Heartfelt thanks to my cousins, whose thoughtful advice and genuine interest kept me motivated along the way. To all the friends, classmates, mentors, and everyone whose names cannot be listed here but whose guidance, constructive criticism, and moral support have been invaluable — your contributions have truly made this journey possible.

Abstract

The exponential growth of Big Data, the Internet of Things (IoT), and Large Language Models (LLMs) has significantly increased the computational and energy demands of modern computing systems. In response, research has increasingly focused on alternative machine learning paradigms and hardware architectures that reduce model complexity and computational load. One such paradigm is the Kolmogorov-Arnold Network (KAN), a novel neural architecture that replaces conventional trainable weights with B-spline-based activation functions.

This thesis presents one of the first comprehensive hardware implementations of KAN inference. To maximize performance, several algorithmic optimizations are introduced, including quantization techniques for static grid B-splines and simplified Look-Up Tables (LUTs) for basis function evaluation. An inference algorithm is developed to exploit the inherent sparsity of B-spline activations through dynamic coefficient bypassing, substantially reducing both memory bandwidth requirements and the number of operations.

On the hardware side, the design features a dedicated Comparator Chain for grid interval detection, a memory layout optimized for sequential coefficient access, and a Processing Element (PE) architecture for first-order B-spline evaluation. The full system is implemented as a custom accelerator integrated with a RISC MicroBlaze processor on a Xilinx Artix-7 FPGA. Experimental results confirm functional correctness and demonstrate trade-offs in performance, area, and scalability.

This work lays the foundation for future exploration of KAN-based models in resource-constrained hardware, offering a scalable platform for spline-driven machine learning.

Popular Science Summary

Modern Artificial Intelligence (AI) breakthroughs have been spectacular. For example, DeepMind’s AlphaFold can predict protein structures and large language models like ChatGPT can write essays - but these advances come at a cost. Training GPT-3 reportedly consumed 1287 megawatt-hours of electricity (about the annual usage of 120 homes). Even running such models on demand uses lots of power: a single ChatGPT query has about $5\times$ the energy footprint of a simple Google search. In short, today’s AI is powerful but power-hungry.

Architectural innovations have played a crucial role in advancing the performance of state-of-the-art AI models while keeping costs manageable. Different architectures offer unique advantages and are optimized for specific types of tasks. For instance, the Multi-Layer Perceptron (MLP) - a network of neural layers connected by millions of weights - serves as the backbone for many AI systems, particularly in function approximation and tabular data processing. Convolutional Neural Networks (CNNs) excel at handling spatial data, making them ideal for applications like image classification and video analysis.

Kolmogorov-Arnold Networks (KANs) offer a fresh perspective. The KAN architecture replaces each weight in a traditional MLP with a tunable single-input function along that connection. In other words, every “weight” becomes a smooth mathematical curve that the network learns, rather than a single number.

This work trains KANs on image-classification tasks and designs a custom hardware accelerator to run them, aiming to see whether KANs may compete with traditional neural networks by reducing parameter count and energy cost. The accelerator exploits a unique KAN characteristic - only small segments of each curve are relevant for any given input - enabling it to bypass unnecessary computations, thereby saving processing time and energy. Further optimizations, like using look-up tables, led to a compact design that can theoretically scale in parameter count without increasing the computation cost.

From identifying optimal architectures for KANs in image classification to applying algorithmic optimizations and designing custom hardware capable of running on actual silicon, this work presents one of the first comprehensive investigations into KANs and their efficient implementation. The insights and innovations shared here aspire to drive further advances at the intersection of algorithm and hardware design, ultimately supporting smarter, faster, and more energy-efficient systems.

Table of Contents

1	Introduction	1
1.1	Motivation	3
1.2	Contributions	4
1.3	Related Work	5
1.4	Thesis Outline	6
2	Background	9
2.1	Kolmogorov-Arnold representation Theorem	9
2.2	Basis Splines	10
2.3	Kolmogorov-Arnold Networks	13
3	The Algorithmic Perspective: From Training to Inference	17
3.1	Training	17
3.2	Quantization	26
3.3	Inference Algorithm	29
4	The Hardware Perspective: From Inference to Silicon	31
4.1	Architecture Overview	32
4.2	AHB & System Control Unit	34
4.3	Memory System Design	34
4.4	Matrix-Vector Multiplication Unit	36
4.5	Compute Control Unit	37
4.6	FPGA Target and MicroBlaze	40
5	Results and Discussion	43
5.1	Validation	44
5.2	Timing and Performance	44
5.3	FPGA Utilization	46
5.4	Power	47
5.5	Summary	48
6	Conclusion	49
7	Future Work	51

References	55
A Extra material	57

List of Figures

1.1	Comparison of KAN and MLP architectures.	2
1.2	The KAN Pipeline.	4
2.1	Graphical explanation of spline notation.	10
2.2	Process of curve fitting using B-splines.	11
3.1	Training KAN for xy dataset.	19
3.2	Comparison of basis functions based on grid uniformity.	20
3.3	Difference between sampling and quantization.	26
3.4	Sampled first-degree basis functions.	27
3.5	Usage of LUT for AF evaluation.	29
4.1	KAN accelerator system architecture block-diagram.	33
4.2	KAN accelerator memory map.	34
4.3	A-RAM regions.	35
4.4	Layout of the C-RAM.	36
4.5	MVME block diagram.	36
4.6	PE block diagram.	37
4.7	RQ block diagram.	37
4.8	CCU FSM - IDLE and SETUP states.	39
4.9	CCU FSM - EVAL state.	40
4.10	CCU FSM - OUTPUT_WB state.	41
5.1	Breakdown of dynamic power consumption by accelerator submodules.	47
A.1	AHB KAN peripheral in a MicroBlaze system.	58

List of Tables

1.1	KAN implementation results for Dry Beans Dataset	5
1.2	Comparison of KAN and DNN ICs.	6
3.1	Summary of KAN training on xy regression task.	18
3.2	Summary of KAN training on MNIST dataset.	22
3.3	Summary of several MLP models trained on MNIST.	25
3.4	Full-Grid LUT	28
3.5	Half-Basis LUT	29
5.1	KAN models used for performance evaluations.	43
5.2	Performance metrics for the reference KAN models.	46
5.3	Utilized FPGA resources divided by type.	47
5.4	Accelerator specifications.	48

List of Acronyms

A-RAM Activations RAM

ADAM Adaptive Moment Estimation

AF Activation Function

AHB Advanced High-performance Bus

AI Artificial Intelligence

ASCU AHB & System Control Unit

BNN Binary Neural Network

BRAM Block RAM

C-RAM Coefficients RAM

CC Clock Cycle

CCU Compute Control Unit

CIM Compute-In Memory

CNN Convolutional Neural Network

COD Curse Of Dimensionality

DNN Deep Neural Network

FF Flip-Flop

FG-LUT Full-Grid LUT

FPGA Field-Programmable Gate Array

FSM Finite State Machine

GPT Generative Pre-trained Transformer

HB-LUT Half-Basis LUT

HLS High-Level Synthesis

IC Integrated Circuit

IoT Internet of Things

KAN Kolmogorov-Arnold Network

KAT Kolmogorov-Arnold representation Theorem

LBFGS Limited-memory Broyden Fletcher Goldfarb Shanno

LLM Large Language Model

LUT Look-Up Table

MAC Multiply-Accumulate

ML Machine Learning

MLP Multi-Layer Perceptron

MNIST Modified National Institute of Standards and Technology

MVME Matrix-Vector Multiplication Engine

PE Processing Element

PTQ Post-Training Quantization

ReLU Rectified Linear Unit

RISC Reduced Instruction Set Computer

RQ Re-quantization Unit

RTL Register-Transfer Level

SiLU Sigmoid Linear Unit

SNN Spiking Neural Network

UAT Universal Approximation Theorem

Introduction

The exponential growth of data and the proliferation of interconnected devices have ushered in the era of Big Data and the Internet of Things (IoT). These developments have driven the need for advanced Machine Learning (ML) models capable of processing vast amounts of information in real time [1, 2]. Simultaneously, the rise of Large Language Models (LLMs) like Generative Pre-trained Transformer (GPT) has stressed the computational and energy demands of modern Artificial Intelligence (AI) hardware systems [3, 4]. These technologies, while transformative, pose significant challenges in terms of power consumption, scalability, and computational efficiency.

One of the widely used ML architectures is Multi-Layer Perceptron (MLP), where each layer of neurons is connected through scalar weights and non-linear Activation Functions (AFs). In MLPs, all the linear calculations like multiplication and accumulation can be modeled as matrix multiplications. This approach enables the representation of MLPs using the mathematical tools available from linear algebra. MLPs have become the backbone of today's ML and AI research due to their mathematical scalability, as the user can just increase the number of neurons and layers to reduce error, and inherent non-linearity (through AFs). They are the default model in ML for approximating non-linear functions, due to their power guaranteed by the Universal Approximation Theorem (UAT) [5].

MLPs have several significant drawbacks. First, MLPs suffer from large neural scaling factors - that is, the number of model parameters required to achieve a given reduction in test loss increases exponentially with the number of inputs to the model. More specifically this factor is shown to be around $4/d$, where the d is the dimension of the dataset [6]. This makes MLPs computationally expensive and memory-intensive, particularly for tasks involving high-dimensional data. In addition, MLPs are not inherently interpretable - it is not possible to get an insight into how the model works inside. For example, in the case of AlphaFold [7], the authors were able to train an ML model to accurately predict protein structures, but without fully understanding how the model performs the prediction.

While hardware optimizations can greatly amend the shortcomings of MLPs, higher-level improvements at the algorithmic and architectural levels have an even greater impact. These enhancements may reduce the number of parameters needed

for the same accuracy or lower the computational cost for a given model size. As a result, the pursuit of efficient ML has driven interest in diverse architectural paradigms, including Convolutional Neural Networks (CNNs), Spiking Neural Networks (SNNs), Binary Neural Networks (BNNs) and others.

The architecture explored in this work is based on the seminal paper “KAN: Kolmogorov-Arnold Networks” by Ziming Liu et al., which proposes a novel ML framework inspired by Kolmogorov-Arnold representation Theorem (KAT) [8, 9]. Similar to MLPs, KANs have fully connected structures. However, instead of scalar weights on the edges, KANs employ learnable one-dimensional AFs - parametrized as Basis Splines (B-splines) - directly on the connections between neurons. Unlike MLPs, which apply non-linear AFs at the nodes after summing the inputs, KANs perform only a summation of the transformed signals. A comparison of the two architectures with identical input, hidden, and output layer sizes is shown in Figure 1.1.

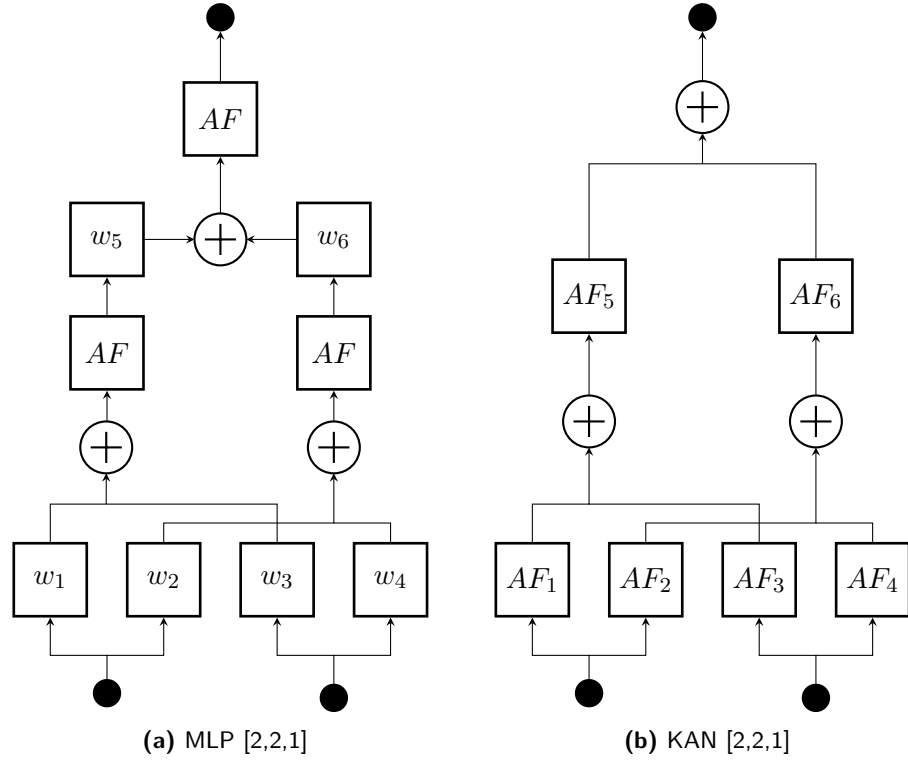


Figure 1.1: Comparison of KAN and MLP architectures with two input, two hidden layer, and one output nodes.

1.1 Motivation

The KAN paper provides a solid mathematical foundation and introduces the core ideas behind this novel architecture. What makes KANs particularly interesting for this thesis is not only their theoretical novelty but also the set of claims made by the authors [10] - claims that this work aims to investigate in the context of hardware acceleration for image classification. These claims include:

1. **KANs can overcome the Curse Of Dimensionality (COD) problem.** The COD refers to the exponential increase in data requirements as input dimensionality grows [11]. Thanks to their mathematical construction, KANs are said to mitigate this issue more effectively than a traditional MLP.
2. **Superior neural scaling laws.** The paper reports that third-order KANs achieve a neural scaling factor of four, significantly outperforming the scaling factors of around one reported for MLPs in previous work [12]. Essentially, a larger scaling factor causes the loss function to approach zero more quickly as model size increases.
3. **Flexible accuracy improvement via grid extension.** KANs offer a unique capability to enhance accuracy through a technique known as grid extension, which involves increasing the number of basis functions that define each AF. This effectively refines the resolution of individual AFs at the edges, potentially boosting accuracy. As a result, KANs can be improved both externally (by increasing depth or width) and internally (by refining individual AFs).
4. **Symbolification of AFs.** After training, the B-spline AFs in KANs can be transformed into symbolic functions. This not only simplifies inference by reducing computational complexity but also enhances the model's interpretability, as the network can ultimately be expressed as a composition of mathematical functions.

Claims one and two suggest that, for a given dataset, KANs may require fewer training samples and fewer weights to achieve the same accuracy as MLP models. Fewer weights could also lead to reduced computational and memory overhead. Claim three offers a way to potentially improve model accuracy without retraining from scratch or changing its inherent structure. Finally, the fourth claim can lead to potential improvement in the computational cost of KAN inference by replacing B-spline AFs with equivalent mathematical functions - provided this substitution does not compromise accuracy. Therefore, the primary goal of this thesis is to evaluate the hardware efficiency of KANs and to design one of the first dedicated KAN accelerators. The focus is on exploring their suitability for edge computing applications, where low power and high performance are critical. This work aims to lay the foundation for future research on deploying KAN-based architectures in real-world, resource-constrained environments.

1.2 Contributions

The contributions span the full stack, from algorithmic improvements and behavioral modeling to hardware design and evaluation on real-world image classification tasks. The key contributions are summarized as follows:

1. **Evaluation of KANs for image classification:** Various network depths, widths, grid sizes, and spline orders were tested to identify architectures suitable for efficient hardware inference without compromising classification accuracy. Modified National Institute of Standards and Technology (MNIST) dataset was used as the main task. MLPs were also trained on MNIST for comparison.
2. **Post-Training Quantization (PTQ) for hardware deployment:** A quantization strategy was implemented to convert trained floating-point KAN models into 8-bit integer representations.
3. **Optimized behavioral model for inference:** A novel behavioral forward-pass algorithm was developed for KANs. The algorithm reduces the computational complexity of KANs, paving the way for efficient hardware realization.
4. **Custom hardware accelerator for KAN inference:** An accelerator architecture was implemented in SystemVerilog, based on the behavioral model. It features an efficient coefficient RAM layout, a comparator chain module for bypassing non-effectual coefficients, and a specialized Processing Element (PE) design optimized for first-order B-splines.
5. **System-level integration on Xilinx Artix-7 FPGA platform:** The architecture was synthesized and implemented on a Xilinx Artix-7 Field-Programmable Gate Array (FPGA), embedded with an AMD 32-bit Reduced Instruction Set Computer (RISC) MicroBlaze soft-core processor.
6. **Open-source research framework for KAN hardware exploration:** All major components - including the training and quantization pipeline, hardware modules, and simulation testbenches - have been modularized and open-sourced to facilitate future exploration and extension by the research community.

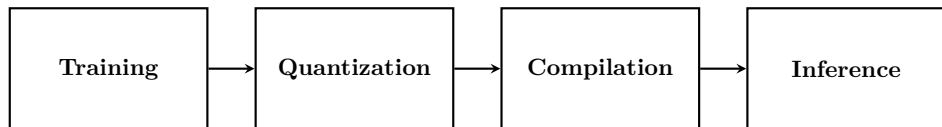


Figure 1.2: The KAN Pipeline.

1.3 Related Work

The computation of B-spline AFs in KANs presents new challenges when it comes to hardware acceleration. The recursive and non-linear nature of B-spline evaluation, complicates acceleration, especially for high-resolution input image and numerous control points.

Since the architecture is new, Integrated Circuit (IC) implementations in the literature are scarce. The first paper to discuss this possibility named *Exploring the Limitations of Kolmogorov-Arnold Networks in Classification*, by Van Duy Tran et al. compared the KAN and MLP for different classification datasets such as Wine, DryBeans, Mushroom and so on. The authors implemented the accelerator in Vitis High-Level Synthesis (HLS) and ran the models on a Xilinx Zynq Ultrascale+ ZCU104 FPGA.

Table 1.1: KAN implementation results for Dry Beans Dataset [13].

Metric	MLP	KAN
Training		
Model Size	16,20,15,10,7	16,2,7
Spline Info	N/A	G=6,k=3
No. Params	892	414
Accuracy	0.921	0.924
Energy Time Product (ET)	0.37	520.56
Power (W)	21.33	22.78
PDP (W*s)	7.892	11353.5
Accelerator Implementation		
Frequency (MHz)	100	100
Power (W)	0.67	14.802
BRAMs	-	781
DSPs	17	9111
FFs	11328	734544
LUTs	8894	1677558
Latency (# Clock Cycles)	835	1896
Latency (Time) (ns)	8350	18960

Table 1.1 indicates that the MLP outperforms the KAN on the Dry Beans dataset across most software and hardware metrics. However, KANs demonstrate a clear advantage in terms of model compactness: they achieve comparable perfor-

mance using less than half the number of parameters, highlighting a more efficient model representation.

It can be noted that the usage of HLS, the lack of parallelization, and hardware-specific optimizations in this study led to poor resource utilization on the FPGA. Therefore, implementing a more efficient inference model offers the potential for improved results. They also didn't quantize the network and carried all the calculations in 32-bit floating-point representation. Despite the negative results, the study serves as a baseline for understanding the computational demands of KANs and highlights areas requiring improvement, such as memory access patterns and efficient B-spline evaluation. The authors also note that KANs require substantially longer training times than MLPs, ranging from 6.55x to 36.68x [13].

Another study, *Hardware Acceleration of KAN for Lightweight Edge Inference* [14], delves more into hardware details, considering the possibility of using Look-Up Tables (LUTs) and Compute-In Memory (CIM) techniques to speed up the inference. Their work, on the other hand, is not tested for image classification datasets. They have shown that for the knot theory dataset, compared to a traditional Deep Neural Network (DNN), their KAN IC achieves far better hardware efficiency, while achieving higher accuracy [14]. Their results are summarized in Table 1.2. These advancements are attributed to architectural optimizations that exploit the repetitive structure of B-spline evaluations, enabling parallel computation and efficient resource utilization.

Table 1.2: Comparison of KAN and DNN ICs [14].

Metric	Improvement
Energy Consumption	51.04x to 77.97x
Area	9.28x to 41.78x
Latency	23.59x to 29.56x

1.4 Thesis Outline

The rest of the thesis is structured the following way:

- **Chapter 2 - Background.** Reviews relevant concepts in neural networks, B-splines, quantization and hardware implementation techniques.
- **Chapter 3 - The Algorithmic Perspective.** Explores training, quantization and the higher-level optimizations for hardware.
- **Chapter 4 - The Hardware Perspective.** Describes the hardware architecture and implementation of proposed KAN accelerator.
- **Chapter 5 - Results and Discussion.** Presents evaluations of the design, measurements in terms of accuracy, latency, throughput and hardware resource utilization.

-
- **Chapter 6 - Conclusion.** Summarizes the findings and gives a final verdict.
 - **Chapter 7 - Future Work.** Outlines possible directions for further research on KAN-based hardware.

Background

This chapter presents the mathematical foundation of KANs, focusing on their use of B-splines for constructing single-variable functions and their optimization via the Limited-memory Broyden Fletcher Goldfarb Shanno (LBFGS) and Adaptive Moment Estimation (ADAM) algorithms. For constructing single-variable functions and their optimization through the LBFGS and ADAM algorithms. Key topics in B-splines are also examined.

2.1 Kolmogorov-Arnold representation Theorem

As mentioned in Chapter 1, KANs are based on KAT. This theory, proposed by Vladimir Arnold and Andrey Kolmogorov [8, 9], establishes that if f is a multivariate function on a bounded domain, then it can be written as a finite composition of continuous functions of a single variable and the binary operation of addition. For smooth $f : [0, 1]^n \rightarrow \mathbb{R}$:

$$f(x) = f(x_1, \dots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right) \quad (2.1)$$

where $\phi_{q,p} : [0, 1] \rightarrow \mathbb{R}$ and $\Phi_q : \mathbb{R} \rightarrow \mathbb{R}$. What the Eq. (2.1) entails is that for a given n variable input function f , it can be dissected as the summation and composition of single-dimensional $\phi_{q,p}$ and Φ_q functions. To adapt this theory for practical use in general ML frameworks, the authors introduce two key modifications [10]:

1. They move beyond the original Eq. (2.1), which assumes a shallow two-layer structure with a limited number of hidden terms $(2n + 1)$, by generalizing the architecture to allow arbitrary depth and width.
2. While the original KAT does not require the one-dimensional functions to be smooth, the authors recognize that non-smooth functions are difficult to learn. To address this, they ensure smoothness by modeling the AFs using B-splines.

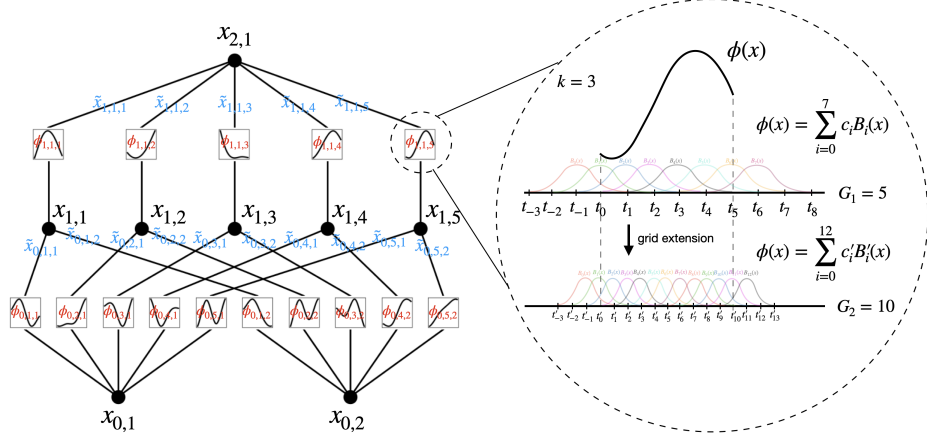


Figure 2.1: Left: Notations of activations that flow through the network. Right: an AF is parameterized as a B-spline, which allows switching between coarse-grained and fine-grained grids [10].

Figure 2.1 illustrates a more detailed view into the KAN architecture. On the right side, it can be seen how a set of basis functions can be scaled and summed to create an AF. In general, splines are used to draw a function given a set of control points. They all connect several points in space more or less continuously, but different types of splines promise different mathematical properties for the generated functions [15]. B-splines, specifically, make sure that the resulting graph is second-degree continuous. This means each section of the spline is connected smoothly - that is the second-degree derivatives at each section connection (knots) are the same value. This property is important for training - as the KAN researchers use LBFGS optimizer to minimize the loss function. LBFGS is a quasi-Newtonian optimization algorithm that uses second degree derivatives to calculate the ideal step direction for loss minimization. Using B-splines ensures that the KAN model is second-degree differentiable, making it suitable for LBFGS. The next section provides a more detailed discussion of B-splines.

2.2 Basis Splines

Visually speaking, B-splines are just a summation of a set of phase-shifted, identical basis functions as can be seen in Figures 2.2a and 2.2b. The example in Figure 2.2 is for a third-order B-spline. To approximate the function shown in Figure 2.2c using the basis functions in Figure 2.2b, a suitable set of coefficients is assigned to each basis function. These coefficients scale the basis functions as illustrated in Figures 2.2d and 2.2e. Summing the scaled basis functions yields the approximation of the original curve, as depicted in Figure 2.2f. The main issue here is finding the correct set of coefficients resulting in the closest approximation

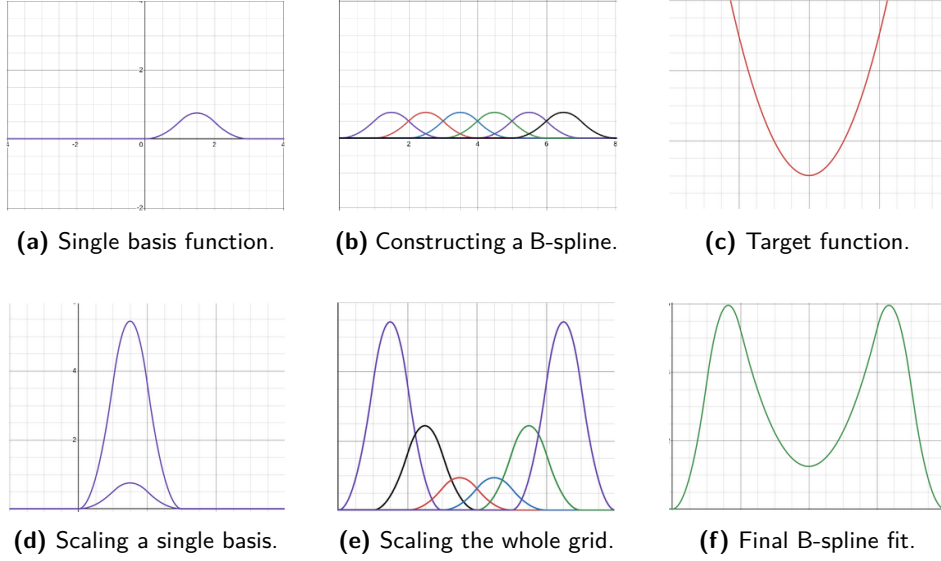


Figure 2.2: Process of curve fitting using B-splines.

of the dataset - it becomes an optimization or training problem.

Mathematically, the basis functions should be defined in such a way that it ensures second-degree differentiability. Cox-DeBoor's recursive algorithm is used to define such function [16, 17]:

$$B_{i,0}(x) = \begin{cases} 1 & t_i \leq x < t_{i+1} \\ 0 & \text{otherwise} \end{cases} \quad (2.2)$$

$$B_{i,k}(x) = \frac{x - t_i}{t_{i+k} - t_i} B_{i,k-1}(x) + \frac{t_{i+k+1} - x}{t_{i+k+1} - t_{i+1}} B_{i,k-1}(x) \quad (2.3)$$

where, $B_{i,k}(x)$ is the value of k -order i^{th} basis-function evaluated at a given point x . Eq. (2.2) defines the formula for zero-order basis function, which is then used recursively to build higher-order basis functions. The input interval (the range of values that x can take) is divided into G number of knot intervals (the grid). In the most basic case of zero-order basis function, an interval of $[t_i, t_{i+1}]$ defines where the corresponding $B_{i,0}$ is non-zero. A k -order basis function $B_{i,k}$ is non-zero over the interval of $[t_{-k+i}, t_{i+1}]$.

In Figure 2.1, the first B-spline has a grid size of $G_1 = 5$; however, the total number of intervals is 11, not 5. This is because, when constructing the B-spline, the knot vector should be extended by $2k$ additional knots, k on each side, to ensure that all basis functions have full support across the entire domain, including at the boundaries. Without this extension, the basis functions near the edges would lose their full shape, leading to reduced continuity and accuracy in those regions.

One might wonder why the grid spacing isn't simply increased to spread the

basis functions more uniformly across the extended input range. However, doing so would reduce the number of intervals within the original input range, thereby lowering the resolution where it matters the most.

Throughout this thesis, besides the grid size (G) and the spline order (k), the following parameters are used extensively:

$$\mathbb{N} = G + k \quad (2.4)$$

$$\mathbb{T} = G + 2k + 1 \quad (2.5)$$

where, $\mathbb{N}$ is the number of basis functions that defines an extended B-spline, and $\mathbb{T}$ is the number of knots that defines the extended grid. These values arise naturally from the grid extension procedure described above.

Figure 2.1 also illustrates the Grid Extension procedure used to increase the resolution of the B-spline within the main input range. In that example, the number of grid intervals is increased from five to ten, resulting in denser basis functions and smaller grid intervals. This procedure is a key component of the KAN architecture and, as discussed earlier, provides KANs with an internal degree of freedom for improvement.

The basis functions are multiplied by their corresponding coefficients and summed to form a B-spline as follows:

$$S(x) = \sum_{i=0}^{\mathbb{N}} c_i B_{i,k}(x) \quad (2.6)$$

Here, for a given x , all basis functions are evaluated and multiplied by their corresponding coefficients. For further exploration, I have created Desmos interactive graphs for first-order¹ and third-order² B-splines. Users can modify parameters and observe how these changes affect the structure and visual appearance of the splines.

Finally, the B-spline parameterized AFs can be represented as symbolic expressions after training, making KANs well-suited for deriving human-understandable mathematical formulas or symbolic laws from data. In ML terms, this ability to understand and explain how a model makes decisions or predictions is called interpretability. The ability of KANs to symbolically represent the entire model or some parts of it is a significant advantage over MLPs, where such symbolic representation is inherently unfeasible [10]. The authors of the paper explores this extensively as well, calling the process Symbolification.

¹<https://www.desmos.com/calculator/mwp3b5jflw>

²<https://www.desmos.com/calculator/bn3jrll1xm>

2.3 Kolmogorov-Arnold Networks

Leveraging ideas of KAT and B-splines, the KAN architecture is defined. The shape of an arbitrary width and depth KAN is represented by an integer array:

$$[n_0, n_1, \dots, n_L] \quad (2.7)$$

A single KAN l^{th} layer with n_{in} -inputs and n_{out} -outputs can be defined as a matrix of 1D B-spline functions [10]:

$$\Phi_l = \{\phi_{q,p}\}, \quad p = 1, 2, \dots, n_{in}, \quad q = 1, 2, \dots, n_{out} \quad (2.8)$$

A general KAN is a composition of L layers and a given input vector of $\mathbf{x}$:

$$\text{KAN}(\mathbf{x}) = (\Phi_{L-1} \circ \Phi_{L-2} \circ \dots \circ \Phi_1 \circ \Phi_0)\mathbf{x}. \quad (2.9)$$

Eq. (2.9) shows that the input $\mathbf{x}$ is passed through the different B-spline AFs starting from input all the way to the output to evaluate the KAN. For comparison, a similar representation for MLP would look like as follows:

$$\text{MLP}(\mathbf{x}) = (\mathbf{W}_{L-1} \circ \text{AF} \circ \mathbf{W}_{L-2} \circ \text{AF} \circ \dots \circ \mathbf{W}_1 \circ \text{AF} \circ \mathbf{W}_0)\mathbf{x}, \quad (2.10)$$

where $\mathbf{W}_l$ denotes the weight matrix for the linear transformation at the l^{th} layer, and $\text{AF}(x)$ represents a pre-set non-linear activation function such as Rectified Linear Unit (ReLU), Sigmoid Linear Unit (SiLU), or others. From Eqs. (2.9) and (2.10), it is evident that while MLPs treat linear and non-linear operations as separate components, KANs combine them within B-spline-parameterized, learnable AFs.

The original KAN paper by Liu et al. [10] extends the architecture by introducing residual AFs alongside trainable B-spline-based AFs. In this formulation, each AF is defined as the sum of a residual component and a spline component:

$$\phi_{q,p}(x) = w_{q,p}^b b(x) + w_{q,p}^S S_{q,p}(x) \quad (2.11)$$

where $b(x)$ is the residual AF, and w_b and w_S are coefficients for residual and spline functions, respectively. The residual function is not trained and generally assigned to one of the traditional AFs used in MLPs (ReLU, SiLU, etc.).

2.3.1 Grid Extension

As mentioned before, one of the notable advantages of spline-based function representations is their capacity for arbitrary approximation accuracy through grid refinement. By increasing the resolution of the underlying grid, the representation can more closely follow the target function. This progressive refinement mechanism provides KANs with a flexible and structured way to scale model complexity, in contrast to the unstructured parameter scaling seen in MLPs.

To describe this process, consider approximating a one-dimensional function f over a bounded interval $[a, b]$ using B-splines of order k . A coarse discretization divides the interval into G_1 segments, generating augmented grid $\{t_{-k}, \dots, t_{-1}, t_0, \dots, t_{G_1}, t_{G_1+1}, \dots, t_{G_1+k}\}$. The function approximation on the coarse grid based on Eq. (2.6) becomes:

$$f_{\text{coarse}}(x) = \sum_{i=0}^{G_1+k} c_i B_i(x) \quad (2.12)$$

where c_i are the learned coefficients.

To transition to a finer resolution with $G_2 > G_1$ grid intervals, a new set of basis functions $B'_j(x)$ is defined on the finer grid. The refined function can then be written as:

$$f_{\text{fine}}(x) = \sum_{j=0}^{G_2+k} c'_j B'_j(x), \quad (2.13)$$

where the new coefficients c'_j are computed to ensure that f_{fine} remains consistent with the original coarse approximation. This process, illustrated in Figure 2.1, allows for seamless model growth and precision enhancement in KANs, without the need for full retraining.

2.3.2 Symbolification

In certain cases, it is reasonable to hypothesize that some learned activation functions in a KAN may correspond to known symbolic expressions, such as x , x^2 , $\cos$ or $\log$. Direct substitution of the symbolic expression f is insufficient, however, because the inputs and outputs of the learned activation function may be subject to affine transformations (e.g., scaling or shifting). To address this, the KAN collects input-output samples - pre-activations x and post-activations y - from the network and fit the symbolic function under an affine transformation. Specifically, the aim is to find parameters $a, b, c, d \in \mathbb{R}$ such that:

$$y \approx c \cdot f(ax + b) + d. \quad (2.14)$$

This approach allows the symbolic form to approximate the learned function accurately, while accounting for possible shifts and scalings in the activation data.

2.3.3 Number of coefficients

As evident from Eq. (2.6), each AF in a KAN has as many coefficients as the number of basis functions used to define it - this value is given in Eq. (2.4). Keeping this in mind, let's assume the following general KAN model:

1. L number of layers
2. $[n_0, n_1, \dots, n_L]$ shape

Subsequently, the number of coefficients for the KAN can be calculated with the following equation:

$$\mathbb{P} = \mathbb{N} \sum_{i=0}^{L-1} n_i \cdot n_{i+1} = (G + k) \sum_{i=0}^{L-1} n_i \cdot n_{i+1} \quad (2.15)$$

In comparison, an MLP network, wouldn't need the overhead caused by the spline coefficients on each edge, it would have only one weight per edge. The authors are aware of this, however they note that in general KANs need less wide networks compared to MLPs [10].

The Algorithmic Perspective: From Training to Inference

This chapter explores the mathematical foundations of KANs in more detail and examines how these principles can inform architectural optimizations across multiple layers. The discussion begins with the model structure and training process, then moves on to quantization strategies, and finally addresses the design of an efficient inference algorithm. By grounding each optimization step in the underlying theory, the aim is to develop a more compact and computationally efficient implementation without compromising accuracy.

3.1 Training

Since the KAN architecture is fairly recent, its training dynamics across various datasets remain relatively underexplored. In particular, there is limited investigation into architectural and training-related optimizations that can reduce computational overhead for the MNIST dataset. This section reviews several techniques aimed at improving the accuracy of KANs while simultaneously minimizing the number of parameters, with a focus on enhancing overall efficiency.

KAN researchers have released a Python library - PyKAN - for interacting with KAN models [18]. Built on PyTorch¹, the library is well-documented and suitable for initial experimentation. It was used extensively throughout this project for training models and gaining a deeper understanding of KANs. However, the library contains many features unnecessary for this work, resulting in added overhead, therefore slow training. A second open-source library - Efficient-KAN - was also employed, offering faster training and better performance for the specific needs of this project. Both libraries have been adapted to varying extents to enhance their performance and accuracy.

¹A widely used open-source framework for building and training deep learning models with GPU support.

3.1.1 Function Regression

A task extensively used by the original authors - a simple mathematical function regression - is the starting point for this project's training experiments as well. Specifically, the goal is to train a KAN to approximate the function xy and subsequently extract a symbolic expression from the trained model. This section is implemented using the PyKAN library.

Table 3.1: Summary of model configurations and results for the xy regression task.

	Before refinement	After refinement	After symbolification
Shape	2, 2, 1	2, 2, 1	2, 2, 1
G	5	7	-
k	3	3	-
Parameter count	108	128	32
Optimizer	LBFGS	LBFGS	-
Base Function	SiLU	SiLU	-
Test Loss	3.23e-04	7.84e-05	5.79e-16

The function to be approximated is defined as follows:

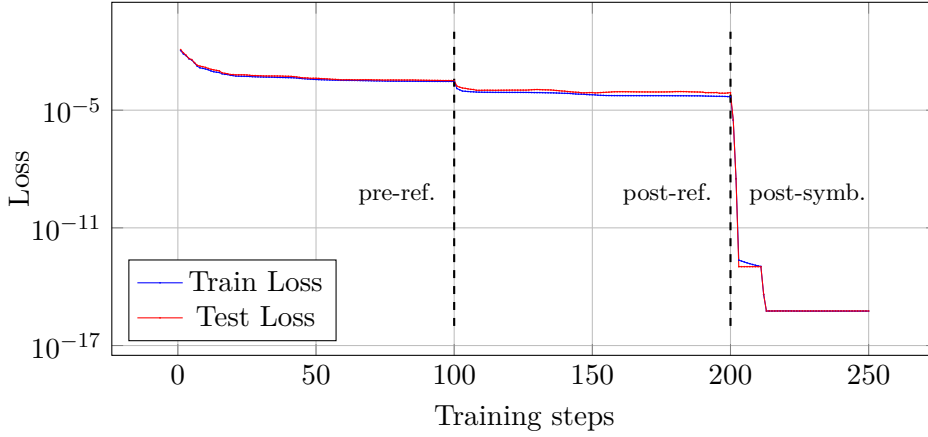
$$f(x, y) = x \cdot y \quad (3.1)$$

where, x and y are the inputs to the function and can take any floating-point values. In this case, both variables are constrained to the range $[-1, 1]$. The dataset is generated by sampling the function using 1000 randomly selected input pairs for training and another 1000 for testing. A KAN model with a (2, 2, 1) architecture is used: the first layer corresponds to the input, with two nodes reflecting the two input variables, and the final layer consists of a single output node representing the function output f . Table 3.1 shows all the parameters of the initial network (pre-refinement¹).

The loss curve before the *pre-ref.* dashed line in Figure 3.1a shows an initial decrease followed by stagnation. At training step 100, the grid size is increased from five to seven, leading to a further drop in loss before it flattens again. To improve performance, a symbolification step is applied near the end (step=200), significantly reducing the loss to near zero. The resulting symbolic expression is:

$$-0.2629 \cdot (0.9508 \cdot x - y)^2 + 0.2629 \cdot (0.9508 \cdot x + y)^2 = 1.0033 \cdot xy$$

¹The terms refinement and grid extension are used interchangeably.



(a) Loss curves for the xy KAN model over all optimization steps.

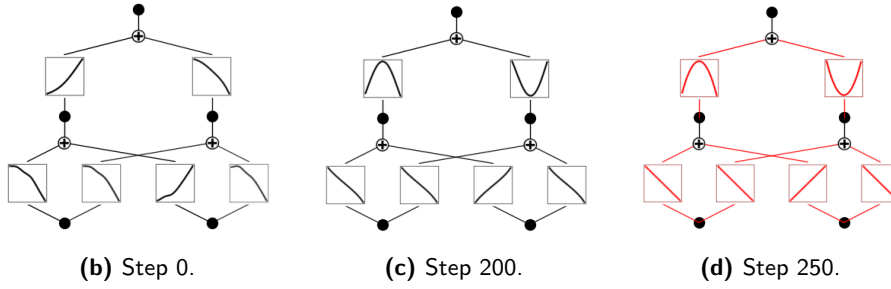


Figure 3.1: Training KAN for xy dataset.

As demonstrated, the KAN model successfully approximates the target mathematical function with high accuracy. Figures 3.1b to 3.1d illustrate the evolution of the model's AFs throughout training. Notably, the two functions in the second layer in Figure 3.1d exhibit quadratic behavior, while the first four AFs remain linear - consistent with the extracted symbolic expression.

One of the key insights from this experiment concerns the parameter count. While Chapter 2 outlines the method for computing the number of parameters via Eq. (2.15), the actual values in Table 3.1 deviate from this theoretical estimation. This discrepancy arises because KANs include additional parameters beyond the B-spline coefficients. These are:

1. Base AF weights
2. Spline AF weights
3. Grid knot values

The base and spline weights correspond to $w_{q,p}^b$ and $w_{q,p}^S$ in Eq. (2.11). The important point here is grid knots, defined by Eq. (2.5), are adapted dynamically

during training based on the input range to each AF. Consequently, each node in each layer may produce different grid range, therefore different knot locations. These values must be stored, as they are essential for fully defining the B-spline-based AFs. Although not explicitly noted by the KAN authors, this distinction is crucial - both for building an accurate inference model and from a hardware overhead perspective - since MLPs do not require these additional parameters.

Finding 3.1

Grid knot values should be stored for generating the basis functions during the inference.

Another key insight emerges when examining the basis functions within a trained AF. As previously discussed, the grid is adjusted based on the input range; however, the knot positions are not uniformly spaced by default. In fact, the PyKAN library adapts the knot distribution by clustering them around regions with higher input sample density - enhancing approximation accuracy where it matters most. This behavior is illustrated in Figure 3.2a.

While this non-uniform placement can improve accuracy, it also increases computational complexity, as each basis function must be re-evaluated for every new input. This observation leads to the first hardware-oriented optimization: **constraining the training procedure to maintain uniformly spaced knots**. Doing so results in identical basis functions within each AF, which simplifies computation and reduces overhead. During the tests, this modification had little influence on the loss value. The effect of the modification is shown in Figure 3.2b.

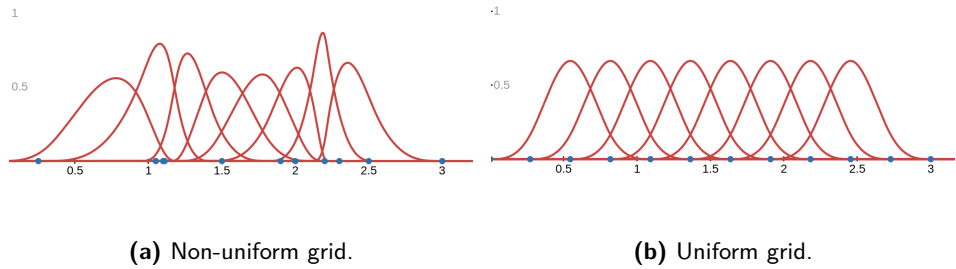


Figure 3.2: Comparison of basis functions based on grid uniformity.

Finding 3.2

KAN models benefit from fixed, uniform grid knot spacing in hardware, as it significantly reduces computational complexity without compromising accuracy.

3.1.2 MNIST

The previous discussion on function regression serves as a foundation for the core focus of this thesis: image classification. This section presents the methodology for training a KAN on the MNIST dataset, with the primary goal of evaluating architectural hyperparameters to identify configurations most suitable for hardware implementation. The hyperparameters explored include:

1. Model shape (depth and width)
2. Grid size (G)
3. B-spline order (k)
4. Base function type (SiLU, ReLU, or none)
5. Grid type (dynamic vs. static)

Careful tuning is required to strike a balance between model accuracy and hardware efficiency. In the following subsections, the effects of varying these hyperparameters are systematically evaluated, with particular emphasis on identifying trade-offs relevant to hardware implementation. It is important to note that all models discussed in this section use a uniform grid.

Model Structure

To identify the optimal KAN architecture - including model shape, grid size, spline order, and base function - for the MNIST dataset dozens of training experiments were conducted. A selection of representative models is summarized in Table 3.2 for clarity and reference.

Due to the inherent non-linearity of KANs, it is theoretically feasible to use a single-layer model, mapping 784 input pixels directly to 10 output classes through B-spline AFs. After tuning the training parameters, this architecture achieved a test accuracy of 93.32%, as shown in Table 3.2. The model comprises 79,968 parameters and uses SiLU as the base function. A breakdown of the parameter count is as follows:

1. B-spline coefficients: $784 \cdot 10 \cdot (5 + 3) = 62,720$
2. Grid knot values: $784 \cdot (5 + 2 \cdot 3 + 1) = 9408$
3. Base function weights: $784 \cdot 10 = 7840$

Note that spline weights are not counted separately, as they can be absorbed into the B-spline coefficients through multiplication before deployment. An interesting observation arises when a hidden layer is introduced and the SiLU base function is effectively removed - replacing it with zero. The resulting model not only achieves a higher accuracy (94.36%) but also uses fewer parameters, as the base function's associated weights are eliminated.

Table 3.2: Overview of various KAN models trained on the MNIST dataset. An asterisk (*) indicates that a static grid was used.

#	Model shape	G/k	Base func.	Parameter count	Test acc. (%)
1	784, 10	5/3	SiLU	79,968	93.32
2	784, 10, 10	5/3	None	73,048	94.36
3	784, 32, 10	5/3	SiLU	238,464	93.86
4	784, 32, 10	5/3	None	213,056	95.81
5	784, 64, 32, 10	5/3	None	430,912	95.79
6	784, 20, 20, 10	5/3	SiLU	156,408	96.44
7	784, 32, 28, 4, 10	5/3	SiLU	245,400	96.07
8	784, 24, 24, 24, 10	5/3	SiLU	192,144	96.35
9	784, 64, 10	5/3*	None	406,528	97.50
10	784, 64, 10	2/1*	None	152,448	97.34
11	784, 24, 10	4/1*	None	95,280	96.66
12	784, 24, 10	2/1*	None	57,168	96.63
13	784, 24, 24, 10	2/1*	None	58,896	95.72
14	784, 128, 10	2/1*	None	304,896	97.90

This result is further supported by a different model configuration featuring 32 nodes in the hidden layer. In this case, the model without any base function outperforms its SiLU-based counterpart by nearly 2% - achieving 95.81% accuracy compared to 93.86% - while also reducing the parameter count by approximately 20,000 (8.4%). The likely explanation is that the MNIST dataset does not benefit from the residual connections (base functions) in KAN. Instead, their presence hinders optimization, making training more difficult and leading to poorer performance. This outcome is especially promising, as it decreases both memory requirements and computational complexity for MNIST KAN, avoiding the hardware overhead associated with evaluating the SiLU function. In summary:

Finding 3.3

Disabling residual connections - i.e., base functions - in the KAN architecture improves model accuracy while also reducing parameter count and computational complexity.

Afterwards, training continued with a deeper and wider architecture, introducing a second hidden layer, resulting in a KAN model with 64 and 32 nodes in the first and second hidden layers, respectively. This modification nearly doubled

the parameter count to 430,912 compared to the previous model. Despite the increased complexity, test accuracy showed no improvement - slightly declining instead. This outcome highlights that greater depth and width do not necessarily lead to better performance.

To investigate whether the increased width contributed to the drop in accuracy, the layer widths were reduced to 20 nodes each while maintaining the same depth. This adjustment led to one of the highest accuracies observed yet - 96.44%. Notably, the presence or absence of residual connections had minimal impact on the final accuracy for this architecture. Furthermore, increasing the model's depth beyond two hidden layers had minimal impact on accuracy. As shown in Models #7 and #8, no improvement beyond previously achieved results was observed.

Finding 3.4

For KANs trained on the MNIST dataset, increasing the width beyond 32 nodes or the depth beyond two layers impedes convergence during training, resulting in lower accuracy compared to smaller architectures.

All models discussed above were trained with a grid size of five. Varying the grid size showed minimal impact on accuracy. However, increasing it beyond a certain point significantly slowed convergence and led to poorer performance. Attempts to use PyKAN's grid extension functionality also proved unsuccessful, as it introduced numerical instability when applied to models with large input dimensions like MNIST. Similarly, the symbolic expression extraction was not feasible for these models. These limitations likely stem from PyKAN being primarily designed for function regression tasks with lower-dimensional input spaces. Rewriting these components to accommodate high-dimensional data was considered beyond the scope of this thesis, though it remains a promising direction for future work.

Grid Size and Spline Order

The PyKAN library employs a dynamic grid by default. The core idea behind this approach is that, during training, the output ranges of intermediate nodes can shift beyond the initially defined input ranges of the AFs of the next layer. To ensure that AFs remain valid over the evolving input domain, the grid must dynamically extend its range, which involves updating both the knot positions and the corresponding spline coefficients. This mechanism is crucial for maintaining accuracy in deeper KAN models. Without it, successive hidden layers would increasingly push activations outside the original grid range, leading to degraded performance.

However, using a dynamic grid comes with a hidden cost: the basis functions for each AF must be evaluated separately. To assess both this overhead and improve training time, another library - Efficient-KAN - was explored to study the impact of using a static grid. In Model #9, the grid size and spline order were kept the same, but the architecture was modified to include a single hidden layer

with 64 nodes. Surprisingly, this model achieved the best accuracy so far: 97.50%. Although its parameter count is similar to Model #5, its accuracy is significantly higher. A key detail here is that the grid range was fixed to $[-1, 1]$ for all AFs, with uniformly spaced knots. As a result, all AFs share the same set of basis functions. This homogeneity is highly advantageous for hardware acceleration, as a single basis function can be reused across all AFs during inference. A static grid also reduces memory usage, as all AFs share the same knot positions. This eliminates the need to store separate knot values for each function - only a single set is enough.

Finding 3.5

Employing a static grid significantly reduces hardware requirements without compromising model accuracy on the MNIST dataset.

All previous experiments focused on KAN models using cubic B-splines. To evaluate whether lower-order splines could maintain similar accuracy, Model #10 was constructed with a first-order B-spline and a minimal grid size of two. This led to yet another surprising result: the model achieved 97.34% accuracy - only a slight drop - while reducing the parameter count by approximately 250,000. This suggests that for grayscale handwritten digits in the MNIST dataset, the high non-linearity provided by cubic B-splines may be unnecessary. In addition to memory savings, first-order B-splines are significantly less computationally demanding to evaluate, offering further benefits for hardware implementation.

To further reduce the parameter count, narrower KAN architectures with first-order B-splines were explored. Reducing the hidden layer width from 64 to 24 nodes and testing with grid sizes of four and two yielded accuracies of 96.66% and 96.63%, respectively - indicating that grid sizes beyond two have minimal impact on accuracy. Model #12 stands out as the strongest candidate overall, achieving competitive accuracy while using only 57,168 parameters. As a result, this configuration was selected as the primary model for subsequent hardware-oriented evaluations in the remainder of the project.

Finding 3.6

Replacing cubic B-splines with first-order ones significantly reduces parameter count and computational complexity, while maintaining comparable accuracy on the MNIST dataset.

While static grids offer memory savings, they come with a trade-off - particularly as model depth increases. Deeper KAN architectures rely more on grid adaptivity to maintain accuracy, and using a static grid can lead to performance degradation. This effect is empirically observed in Model #13 in Table 3.2.

Table 3.3: Summary of several MLP models trained on MNIST dataset.

#	Model shape	AF	Parameter count	Test acc. (%)
1	784, 20, 20, 10	SiLU	16330	96.23
2	784, 30, 20, 15, 10	SiLU	24645	96.98
3	784, 64, 32, 16, 10	SiLU	53018	97.60
4	784, 40, 40, 10	SiLU	33450	97.83
5	784, 40, 40, 10	ReLU	33450	97.22
6	784, 126, 126, 10	SiLU	116182	98.22

Multi-Layer Perceptron Comparison

Several MLP models were trained to compare their performance with that of KAN models, focusing primarily on whether KANs could match or exceed the accuracy of MLPs using the same or fewer parameters. The results, summarized in Table 3.3, generally favor MLPs for the MNIST dataset.

The highest accuracy achieved by a KAN model was 97.90%(Model #14), using a single hidden layer with 128 nodes, a grid size of two, and a spline order of one. In contrast, the best-performing MLP model reached 98.22% (Model #6) while using nearly $2.5\times$ fewer parameters. In most other cases, KANs either required more parameters to match an MLP’s accuracy or performed worse at a comparable parameter count.

However, KANs demonstrated advantages in architectural metrics, like model depth and width. Acceptable accuracy was achieved using just one hidden layer, while all tested single-layer MLP models failed to perform competitively.

It is also worth noting that MLPs incur computational overhead not only from weight multiplications but also from activation function evaluations. Functions like SiLU, for instance, may involve costly mathematical operations if not optimized. Combined with deeper MLP architectures, this can potentially slow down inference, making larger KAN models more competitive in practical scenarios.

Finding 3.7

MLPs incur overhead from both weight multiplications and activation functions like SiLU, while KANs require only activation evaluation. In deeper networks, this can slow inference, making larger KANs potentially more efficient in practice.

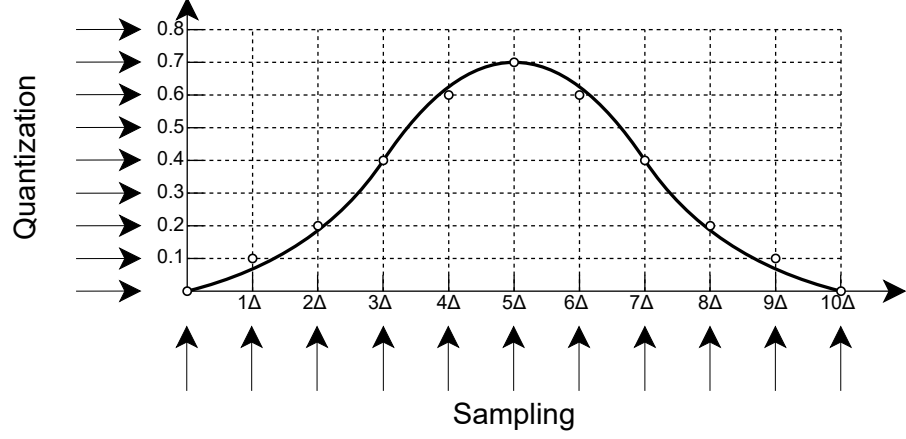


Figure 3.3: Difference between sampling and quantization.

3.2 Quantization

Quantization of KANs is a crucial step in the process of deploying these networks on hardware platforms. The quantization process involves mapping the floating-point values of the network’s weights and activations to discrete levels, which can significantly reduce the memory footprint and computational requirements of the model. This section will discuss the quantization techniques used in this work, as well as the impact of quantization on the performance of KANs.

Since quantization strategies for KANs have not been explored in the literature, a new method was developed. A key idea is that B-spline-based AFs must undergo both sampling and quantization - similar to an analog-to-digital conversion process. This involves discretely sampling the basis functions and then quantizing the sampled values. A conceptual analogy from signal processing is illustrated in Figure 3.3.

In this context, sampling refers to discretizing the x-axis - i.e., the input node values to the AFs. Quantization, on the other hand, involves assigning discrete integer values to the sampled points of the B-spline basis functions. Drawing from signal processing, it is possible to perform sampling without quantization (as seen in analog sample-and-hold circuits). Quantization occurs when the sampled analog values are rounded or truncated to fit a digital representation.

Building on the conclusions from the Training section, a static grid is assumed for the KAN model. As a result, all AFs share the same set of basis functions. The evaluation of a single AF can therefore follow a consistent procedure:

1. Evaluate all the basis functions at the input value,
2. Multiply each result by corresponding coefficients assigned to the AF,
3. Sum the resulting products to obtain the output.

Since the basis functions are identical across AFs, their values for a given input are the same; only the coefficients differ between AFs. To simplify computation, these basis function values can be precomputed and stored in an LUT, allowing reuse across AFs. Naturally, storing continuous functions in a finite-size LUT requires discretization - i.e., sampling the function at fixed intervals. Therefore, the sampling rate determines the number of entries in the LUT. While the sampled values can be stored in either floating-point or integer format, the sampling process itself is independent of the chosen data representation.

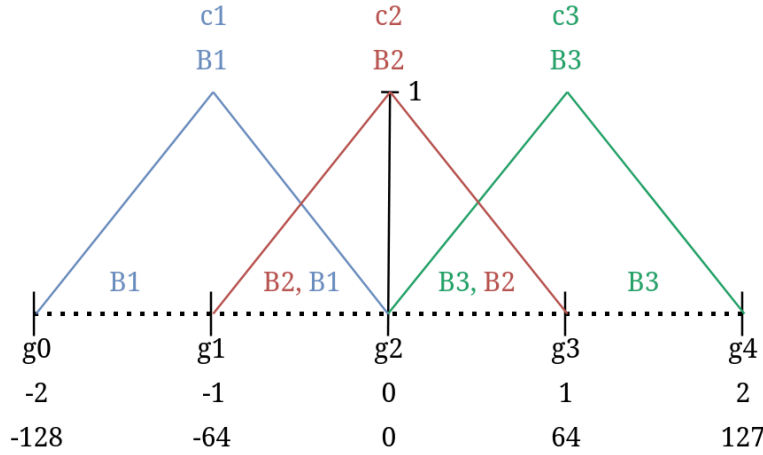


Figure 3.4: Sampled first-degree basis functions.

Figure 3.4 illustrates an example AF composed of three basis functions (B_1 , B_2 , B_3) and their corresponding coefficients (c_1 , c_2 , c_3). The configuration uses a grid size of two and a spline order of one. Grid knots, denoted as g_n , are marked along the x-axis. For each grid interval, the basis functions that are non-zero within that range are indicated.

The main input domain spans from -1 to 1, which - based on the discussion in Chapter 2 - results in an extended grid over the interval $[-2, 2]$. In its original form, this grid would be continuous, allowing the basis functions to be evaluated at any real-valued input using the mathematical formulations in Eqs. (2.2), (2.3) and (2.6).

The figure, however, shows a sampled version of this setup. The dots along the x-axis represent sampled input points. At each sample, the value of all basis functions is computed. For instance, if the grid is sampled at 8-bit resolution, 256 equally spaced samples would be taken across the domain, resulting in 256 precomputed values for all the basis functions. Therefore, in this example the samples taken from the x-axis would be in intervals of:

$$\Delta = \frac{g_{max} - g_{min}}{2^b - 1} = \frac{g_4 - g_0}{2^b - 1} = \frac{2 - (-2)}{2^8 - 1} = 0.015686 \quad (3.2)$$

Starting from -2 (g_{min}), the basis functions can be sampled and stored in an LUT, as illustrated in Table 3.4. Each sampled input value is assigned an

Table 3.4: Full-Grid LUT

Index	Corresponding Input Value	$B_1(x)$	$B_2(x)$	$B_3(x)$
-128	-2	0	0	0
-127	$-2+\Delta$	0.015686	0	0
-126	$-2+2 \cdot \Delta$	0.03137	0	0
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
-30	-0.4628	0.462745	0.537255	0
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
28	0.447	0	0.552941	0.447059
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
127	2	0	0	0

index, facilitating efficient hardware implementation via array access. A signed 8-bit indexing scheme is adopted to simplify the mapping from signed floating-point inputs to integer indices. Importantly, the sampled values in the LUT do not need to be quantized at this stage. The number of columns in the LUT - each corresponding to a basis function - is determined by Eq. (2.4). Consequently, increasing the grid size or the spline order leads to a linear growth in the size of the LUT. This LUT scheme is named Full-Grid LUT (FG-LUT), as it stores the values corresponding the whole grid in the memory.

An illustrative diagram of FG-LUT-based AF evaluation is provided in Figure 3.5. Given an input x , the corresponding precomputed basis function values are retrieved. These values are then multiplied by their associated coefficients and summed to produce the final output of the AF, effectively emulating the procedure described in Eq. (2.6).

After sampling, the floating-point values stored in the FG-LUT can be further quantized into 8-bit integers. This process follows a standard quantization approach, where a scaling factor is computed based on the range of values. Each floating-point entry is then scaled and converted to its corresponding integer representation. For example, in the quantized FG-LUT, the value of 0.015686 would be represented as 2, and 0.03137 as 4, and so on.

The primary concern with using an FG-LUT is its memory complexity. As noted earlier, its size increases row-wise with the number of input samples and column-wise with both the grid size and spline order. Consequently, the overall

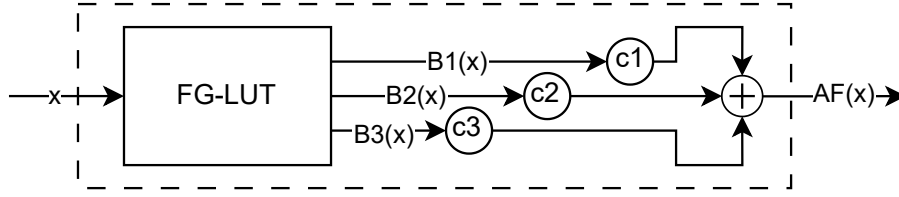


Figure 3.5: Usage of LUT for AF evaluation.

memory complexity is $O(2^b \cdot (G + k))$, making the approach less practical for networks with large grid sizes.

To address this issue, the following solution is proposed. As seen in Figure 3.4, all basis functions are identical due to the use of a static and uniform grid. Additionally, each function exhibits perfect symmetry. These properties are exploited by storing only half of a single basis function and reconstructing the remaining functions in the AF through simple replication or mirroring based on the input. The resulting design is referred to as the Half-Basis LUT (HB-LUT), as illustrated in Table 3.5. In this approach, the number of rows per LUT is reduced to the number of samples divided by the total number of grid intervals (four in the given example), and only a single column of data needs to be stored. As a result, the memory complexity is significantly reduced to $O(\frac{2^b}{G+2k})$.

Table 3.5: Half-Basis LUT

Index	$B_H(x)$
0	0
1	2
2	4
$\vdots$	$\vdots$
63	127

3.3 Inference Algorithm

In addition to the previously discussed optimizations, one more key improvement is introduced. For a given input value, not all basis functions contribute to the evaluation of the AF. In fact, for a uniform-grid B-spline of order k , only $k+1$ basis functions are non-zero and must be evaluated; the remaining functions evaluate to zero. This behavior is illustrated in Figure 3.4, where the active basis functions are shown above their respective grid intervals.

By designing the AF evaluation algorithm to account for this property, the computational complexity can be decoupled from the total number of basis functions $G + k$. This is a significant improvement, as it allows the grid size, and therefore the number of basis functions, to be increased without incurring additional computational cost. This technique, referred to as coefficient bypassing, represents one of the key optimizations proposed in this thesis for efficient B-spline evaluation.

When combined with the HB-LUT technique discussed earlier, these optimizations jointly reduce both the computational load and the memory complexity associated with KAN inference. Based on these improvements the following algorithm for evaluating a single AF is proposed:

1. **Identify the active grid interval.** Given an input value, determine which grid interval it falls into. The intervals are indexed from 1 to $G+2k$, covering the full extended domain of the AF. If the result is zero, the input lies outside the valid range.
2. **Locate the surrounding knots.** Using the identified interval, retrieve the left (lower) and right (upper) grid knot values that bound the input.
3. **Evaluate effectual basis functions.** Compute the indices for the relevant HB-LUT entries. The first basis function is obtained by subtracting the left knot from the input. The second is computed by subtracting the input from the right knot (note the reverse direction - this enables mirroring of the stored half-basis). These two values represent the only non-zero basis functions required.
4. **Apply the corresponding coefficients.** Fetch only the coefficients associated with the active basis functions, multiply them by their respective basis values, and sum the results to obtain the AF output.

A minor simplification in the above algorithm concerns the edge grid intervals - the first and the last. In these boundary cases, only a single basis function evaluates to a non-zero value, as shown in Figure 3.4. Accordingly, the algorithm only computes one basis function: by direct replication for the first interval, or by mirroring for the last. This algorithm serves as the foundation for the hardware implementation, which is detailed in the next chapter.

The Hardware Perspective: From Inference to Silicon

This chapter presents the proposed hardware architecture for accelerating KAN inference, building on the foundational concepts introduced in the previous chapter. It begins with a high-level overview of the system, outlining its primary components and data flow. Subsequent sections provide detailed descriptions of each submodule, highlighting their functionality, interfaces, and implementation rationale. Design choices are motivated with respect to performance, area efficiency, scalability, and compatibility with the overall computation model of KANs.

The chosen accelerator architecture follows an output-stationary dataflow model, where partial sums are retained in place until the corresponding output node has been fully computed. In this approach, input features and coefficients are streamed sequentially through the datapath, while the outputs remain fixed in local storage (e.g., registers of PEs). Once a node's output (or a group of nodes, depending on the parallelism level) is fully computed, the architecture advances to the next node by reloading the inputs and coefficients from the start. This method minimizes write-back traffic for outputs.

Listing 4.1 illustrates the nested loop structure used in the KAN accelerator to evaluate activations across all layers. The two innermost `parfor` loops capture the accelerator's two-level parallelism. The innermost loop over `k` reflects the number of Multiply-Accumulate (MAC) units per PE. Unlike conventional accelerator designs, where each PE typically contains a single MAC, each KAN PE includes two MAC units. This design choice stems from the fact - discussed in the previous chapter - that at most $k + 1$ (here, two) non-zero basis functions are involved in evaluating a first-order B-spline activation function.

The outer `parfor` loop over `q2` spans all the PEs, which is fixed to four in the current architecture. Combined, the loops over `q1` and `q2` cover the entire layer output size Q_1 . Since the number of PEs may not divide Q_1 evenly, the last iteration might leave some PEs idle, reducing overall utilization. For example, if $Q_1 = 10$ and $P = 4$, the loop will iterate three times, producing up to four outputs per iteration. However, in the final iteration, only two PEs will be active.

Listing 4.1: Nested loops description of the KAN accelerator.

```

1 for l in [0, L) do
2   for q1 in [0, ⌈Q1/P⌉) do
3     for d in [0, D) do
4       for i in [0, I1) do
5         parfor q2 in [0, P) do
6           q = q1*P + q2
7           parfor k in [0, 2) do
8             o1[d,q] += c1[q,i,k] * b1[d,i,k]
9           end parfor
10        end parfor
11      end for
12    end for
13  end for
14 end for

```

The surrounding loops iterate over input images in a batch (d) and layer input dimensions (i), accumulating the weighted basis function evaluations into the output tensor o_1 . Each layer l is processed sequentially.

4.1 Architecture Overview

The high-level system architecture of the proposed KAN accelerator, along with its submodules, is shown in Figure 4.1. The architecture comprises several key components:

1. AHB & System Control Unit (ASCU) - Decodes Advanced High-performance Bus (AHB) transactions, selects submodules based on the address, stores the accelerator's status (READY/ BUSY), and triggers the start of the computation.
2. Matrix-Vector Multiplication Engine (MVME) - Performs the core computations and stores intermediate results until the outputs are ready.
3. Coefficients RAM (C-RAM) - Holds the coefficients of the KAN model in the required evaluation order.
4. Activations RAM (A-RAM) - Stores input images, intermediate activations, and the final output.
5. Compute Control Unit (CCU) - Manages access to on-chip RAMs, ensuring correct data flow to and from the MVME.

All multiplexers in Figure 4.1 manage access to the on-chip RAMs and configuration registers, arbitrating between the CCU and the external AHB master. When the `ccu_ready` signal is low - indicating the accelerator is BUSY - the CCU is granted priority access to all memories. Otherwise, the AHB master is allowed

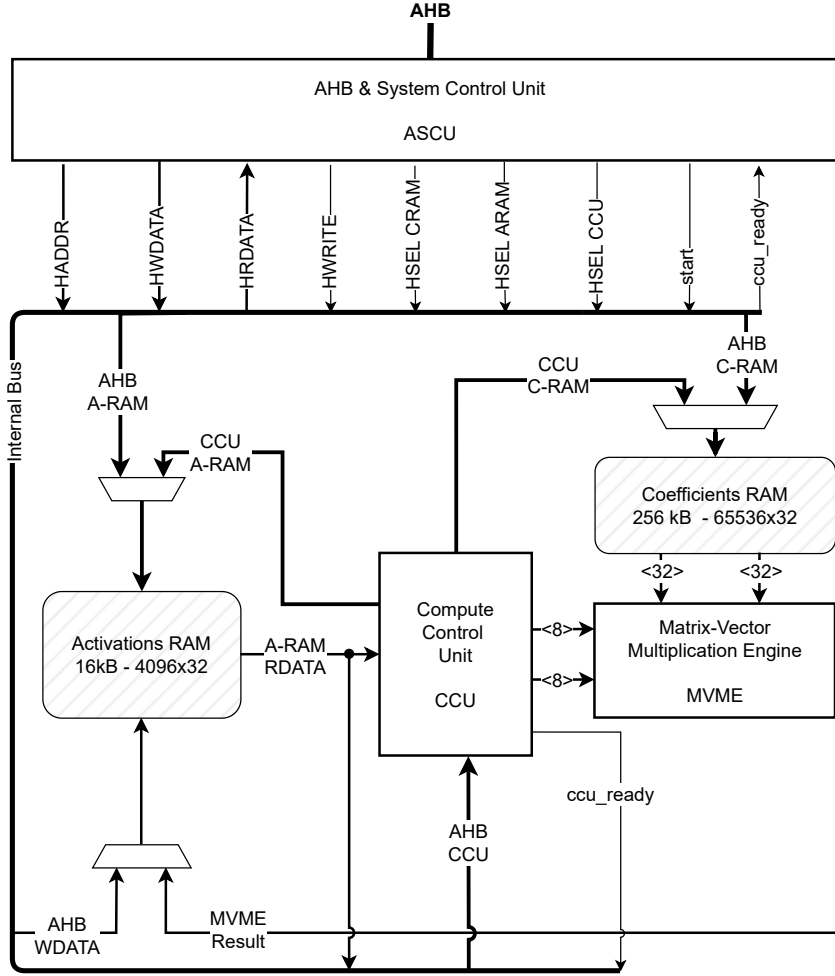


Figure 4.1: KAN accelerator system architecture block-diagram.

to access and load data into the memories. Among these, only the A-RAM supports read operations; all other memories and registers are write-only. This design choice simplifies the implementation of the ASCU.

At a higher level of operation, the AHB master first loads the A-RAM, C-RAM, and configuration registers. It then initiates computation by setting the start bit in the control register. The master waits until the status bit in the status register is set to one, indicating the computation is complete. An example C code illustrating this sequence is provided in Listing A.1 in Appendix A.

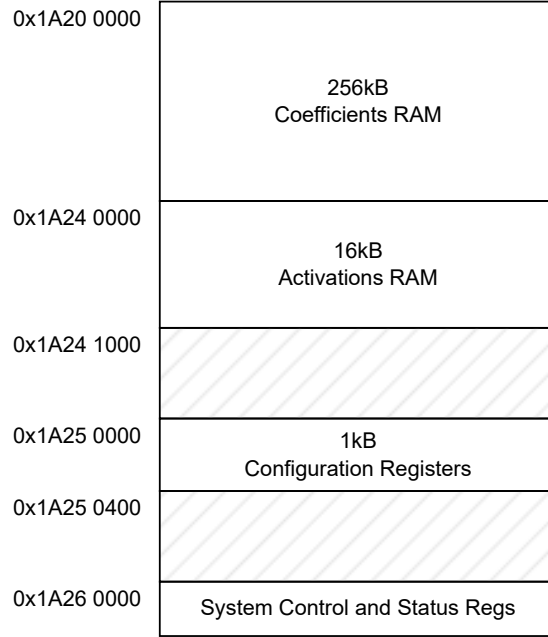


Figure 4.2: KAN accelerator memory map.

4.2 AHB & System Control Unit

This module manages AHB communication with external AHB master and oversees the control and status registers. The inputs and outputs of the ASCU module are illustrated in Figure 4.1. It decodes incoming AHB addresses based on the memory map shown in Figure 4.2 and activates one of the internal HSEL signals to route data to the appropriate subsystem through the Internal Bus.

When the start bit in the control register is set to one, ASCU asserts the start signal, notifying the CCU that all necessary data has been loaded and computation can begin. At the same time, it clears the status bit to zero. It then continuously monitors the `ccu_ready` signal, and once it is asserted, the status bit is set to one, indicating that the computation is complete.

4.3 Memory System Design

The accelerator features two on-chip memories: Activations RAM and Coefficients RAM. The A-RAM stores both input images and results from the PEs. It has a size of 16kB with a 32-bit data width, allowing each row to hold four 8-bit values. In theory, this capacity is sufficient to store a batch of 10 MNIST images along with the output of a single layer containing up to 819 nodes. The C-RAM, on the other hand, is 256kB and is designed to accommodate KAN models with up to approximately 250,000 coefficients.

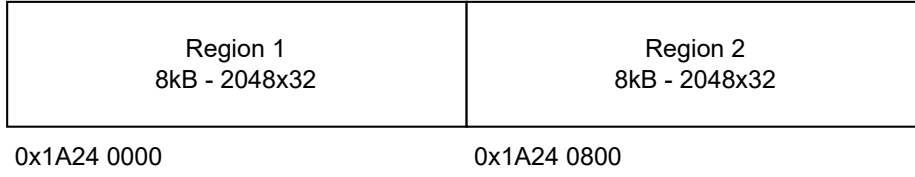


Figure 4.3: A-RAM regions.

To enable the use of A-RAM for both input feature maps and layer outputs, it is divided into two separate regions: Region 1 and Region 2, as shown in Figure 4.3. The AHB master initially loads the MNIST images into Region 1. When computation begins, the CCU reads inputs from Region 1 and writes the layer outputs to Region 2. After each layer is evaluated, the roles of the regions are swapped - Region 2 becomes the input for the next layer, and Region 1 is used for outputs. This alternating pattern continues throughout the inference process. As a result, the final output will reside in Region 1 if the number of layers is even, and in Region 2 if it is odd. The AHB master must be aware of this and read from the correct A-RAM region depending on the number of layers.

4.3.1 Memory layout of the Coefficients RAM

The coefficient layout in C-RAM is designed to minimize additional addressing logic. Each row stores four 8-bit coefficient values, each corresponding to a different output node. The coefficients required for a single AF evaluation are stored sequentially. According to Eq. (2.4), a B-spline of order k with a grid size G requires $G + k$ basis functions - and thus $G + k$ coefficients. This structure is illustrated in the top-left corner of Figure 4.4.

Each set of $G + k$ rows for one AF is repeated I times, once for each input dimension of the layer. These are then grouped across $\lceil Q_1/4 \rceil$ segments, covering all output nodes in batches of four. If Q_1 is not divisible by four, zero-padding is added to complete the final group. Finally, all these segments are stacked across L layers to fully populate the C-RAM with all the coefficient tensors, each having dimensions $[Q_1, I_1, G+k]$. This layout ensures compatibility with the accelerator's dataflow while maintaining efficient memory access.

As discussed earlier, for a B-spline with grid size G , at most two basis functions are non-zero for any given input. This means that out of the $G + k$ coefficients, only two are effective - the rest are multiplied by zero and therefore ineffectual. To evaluate an AF in one CC, two rows from the C-RAM must be accessed simultaneously. Consequently, the C-RAM needs to support dual-port operation. Since these two rows are always consecutive, instead of implementing a true dual-port RAM, two single-port RAMs are banked to emulate a dual-port RAM. The memory is split into two banks: one holding values at even addresses and the other holding values at odd addresses and read-mode bits control which outputs are routed where.

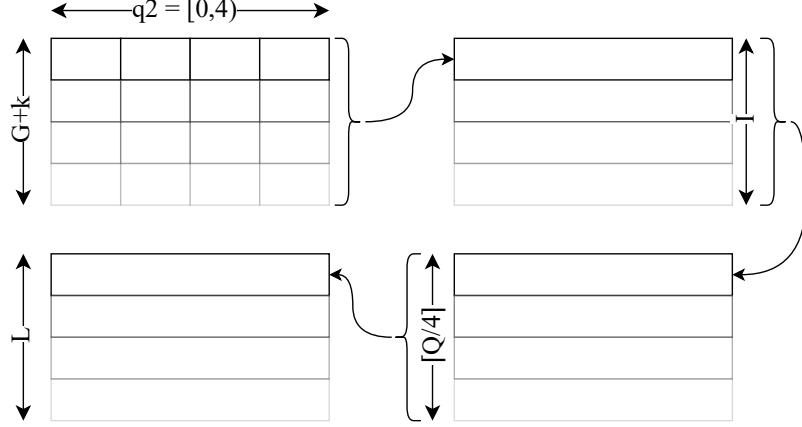


Figure 4.4: Layout of the C-RAM.

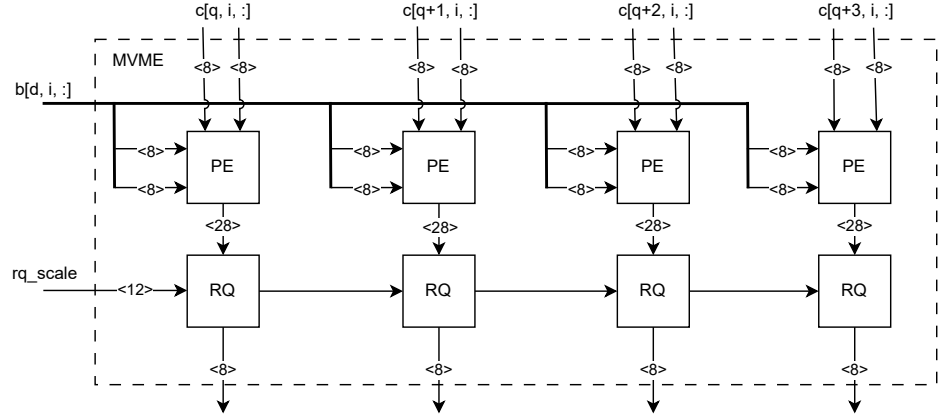


Figure 4.5: MVME block diagram.

4.4 Matrix-Vector Multiplication Unit

The MVME is the core computation unit where actual operations are performed. As shown in Figure 4.5, it comprises four PEs and their corresponding Re-quantization Units (RQs). The internal structure of a single PE is illustrated in Figure 4.6. As previously mentioned, each PE contains two MAC units to handle the two effectual coefficients associated with each AF. The 8-bit input values are multiplied and accumulated in a 28-bit register, which was empirically verified to be sufficient to avoid overflow during accumulation.

The RQs are responsible for scaling the 28-bit accumulated values back to 8 bits for use in the next layer. A single re-quantization scale is shared across all RQs, as it remains constant within a layer. The detailed structure of an RQ is shown in Figure 4.7. Notably, the RQ performs several steps beyond simple

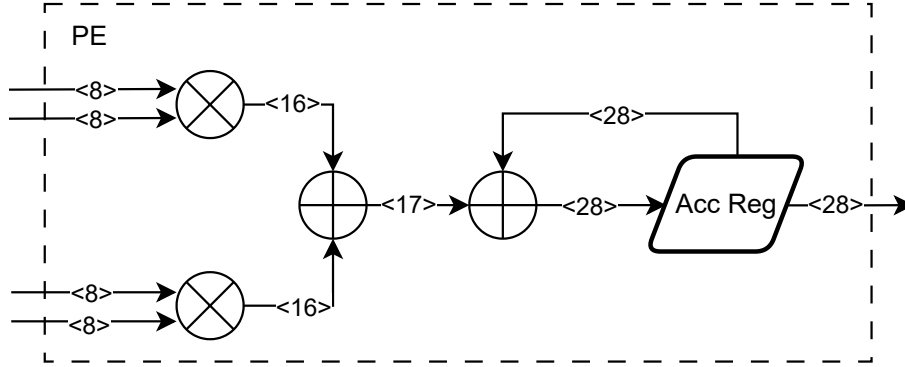


Figure 4.6: PE block diagram.

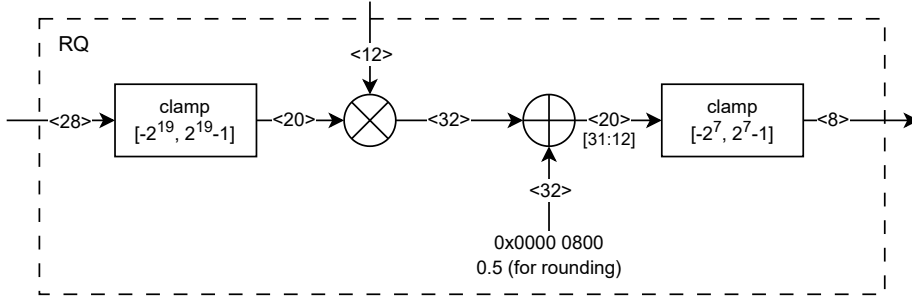


Figure 4.7: RQ block diagram.

scaling. First, it clamps the 28-bit input to a signed 20-bit value to reduce the size of the multiplier. After multiplication, a rounding constant—representing 0.5 in fixed-point with 12 fractional bits—is added. This ensures that the result remains rounded to the nearest integer when right-shifted by 12 bits. Finally, the output is clamped to a signed 8-bit range to ensure it remains within representable bounds.

4.5 Compute Control Unit

The CCU is arguably the most critical component of the accelerator. It orchestrates access to both A-RAM and C-RAM, handles basis function evaluation, and ensures correct sequencing of operands and computations. Internally, it operates as an Finite State Machine (FSM) with four states:

1. IDLE: The default state where the CCU waits for the start signal. In this state, all internal flags and counters are reset.
2. SETUP: Prepares the system by aligning the timing of input data and coefficient streams, accounting for pipeline delays between A-RAM and the compute units.

3. EVAL: Performs the actual computation of the non-zero basis functions using the relevant inputs and coefficients.
4. OUTPUT_WB: Once all inputs for the current output set have been processed, this state handles writing the results back into A-RAM.

4.5.1 Comparator Chain

A key component of the CCU is the Comparator Chain, whose implementation is shown in Listing A.2 in Appendix A. This unit determines the grid interval in which the current input value (fetched from A-RAM) resides. Based on the input's position relative to the grid, the interval index can take on the following values:

- **0**: Indicates the input is outside the grid range. Both basis functions are therefore zero.
- **1**: Indicates the input lies in the first grid interval. In this case, **base2** is zero, while **base1** is computed using values from the HB-LUT.
- **$G + 2k$** : Denotes the input falls in the last grid interval. Here, **base1** is zero, and **base2** is obtained from the HB-LUT.
- **All other values**: The input is within an interior grid interval, so both basis functions are evaluated using the HB-LUT.

As will be explained, this index is used by the CCU FSM to determine which specific coefficients to fetch from C-RAM - limited only to the effectual ones - thereby reducing the computational complexity of KAN inference.

4.5.2 Algorithmic State Machine with Datapath

The IDLE state of the CCU state machine is shown in Figure 4.8. As described earlier, this is the default state where all internal registers and flags are reset. The `ccu_ready` signal is asserted to indicate that the CCU is prepared for computation. Once the start signal is received, the FSM increments the output and batch counters. Notably, the output counter is incremented by four, as the system evaluates four outputs in parallel using four PEs.

Following this, the FSM transitions to the SETUP state. In this state, the `mvme_reset` signal is asserted, prompting the MVME to reset its internal accumulation registers. The FSM remains in the SETUP state for two clock cycles: the first allows A-RAM to output the data at the specified address, and the second allows the comparator chain to determine the grid interval of the fetched input.

Next, the FSM enters the EVAL state, depicted in Figure 4.9. Here, the FSM uses the grid interval index to decide which basis functions to evaluate and which coefficients to fetch from C-RAM. It sets the appropriate C-RAM address and read mode based on the index and the C-RAM layout described earlier in Figure 4.4. The input counter is then incremented, and the C-RAM coefficient

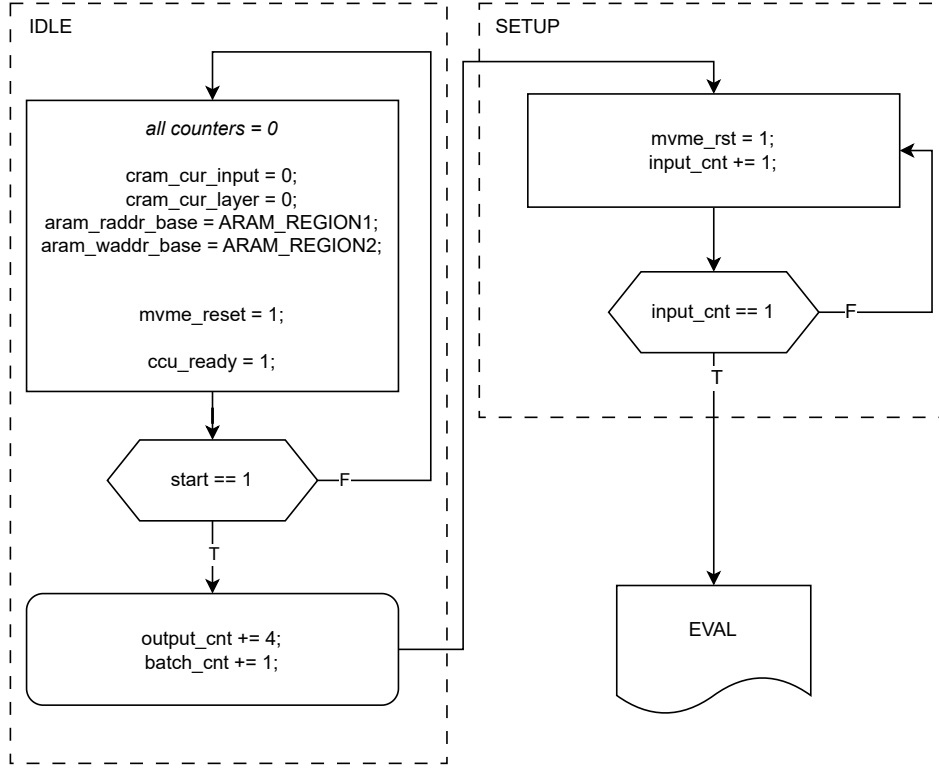


Figure 4.8: CCU FSM - IDLE and SETUP states.

pointer is advanced by $G + k$, which corresponds to the number of coefficients per input for a single AF. The FSM stays in the EVAL state until all inputs for the current image have been processed and four output values are produced.

Once the outputs are ready, the FSM enters the OUTPUT_WB state. In this state, the computed results are written back to A-RAM. The FSM also updates and checks various counters against their predefined limits to determine whether to proceed to the next batch or layer. A key step in this state is the swapping of the A-RAM read and write address base pointers after completing a full layer. This ensures that input and output data are kept separate and correctly aligned for the next layer's evaluation.

In summary, the CCU orchestrates the core functionality of the accelerator by tightly managing memory accesses, coordinating with the MVME, and dynamically adjusting control flow based on input data characteristics. Through its structured state machine and integration with the comparator chain, it ensures that only effectual basis functions and corresponding coefficients are processed, maximizing efficiency. The design leverages techniques like pointer swapping and banked memory access, making the CCU a critical enabler of high-throughput and low-latency KAN inference within the proposed hardware architecture.

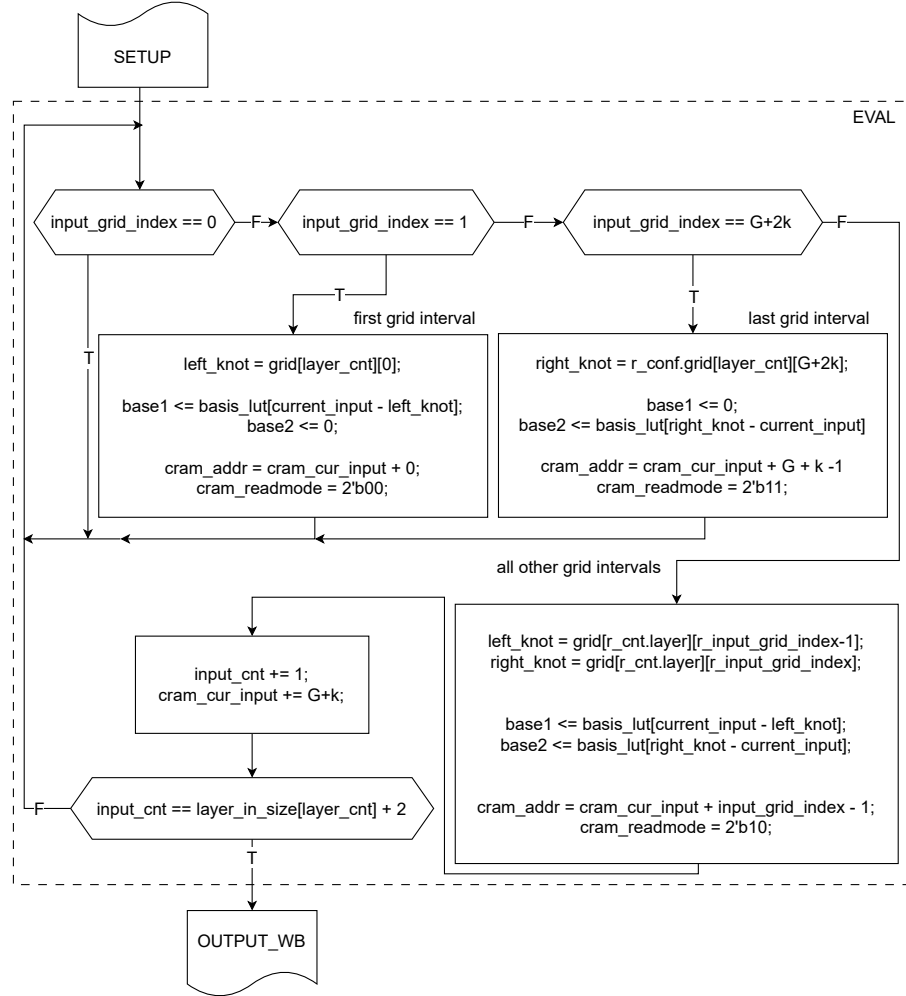


Figure 4.9: CCU FSM - EVAL state.

4.6 FPGA Target and MicroBlaze

The designed accelerator was synthesized and implemented on a Xilinx Artix-7 FPGA, integrated within a larger 32-bit RISC system built around the MicroBlaze soft-core processor. This setup allowed for high-level software control, including data transfer, configuration, and execution monitoring, while the accelerator handled the core computations. The complete system architecture is illustrated in Figure A.1 in Appendix A. Here, the MicroBlaze processor communicates with the accelerator through AXI-to-AHBLite bridge IP, coordinating tasks such as loading input images and coefficients, setting control registers, and reading back the results. Additionally, a UART peripheral is also integrated to support debugging and data logging. This architecture serves as a flexible proof-of-concept platform,

enabling validation of the accelerator with real-world workloads and assessment of its performance across different inference scenarios.

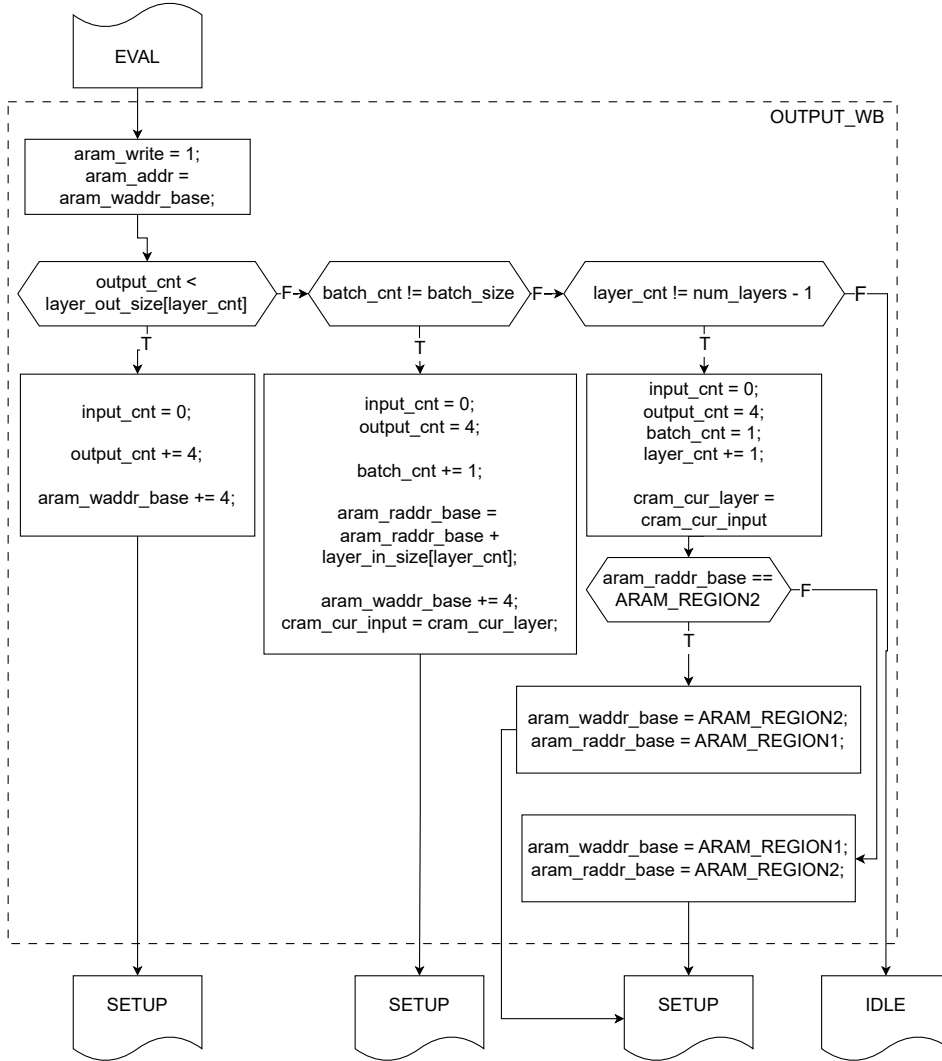


Figure 4.10: CCU FSM - OUTPUT_WB state.

Results and Discussion

This chapter presents the performance and hardware evaluation results of the implemented accelerator, focusing on its ability to execute various KAN models efficiently while maintaining classification accuracy. A total of five distinct KAN configurations were selected for this evaluation, each representing different design trade-offs in terms of width, depth, spline order, and grid size. These models were chosen based on the training results discussed in the previous chapter, balancing accuracy, parameter count, and computational complexity.

For clarity and conciseness, each model is assigned an index, as shown in Table 5.1. These indices will be used throughout the remainder of this chapter when referencing specific models and their associated performance metrics.

The subsequent sections analyze key aspects such as memory footprint, throughput, latency, and accuracy degradation after quantization and deployment. Comparisons between models highlight how different architectural choices influence hardware resource usage and inference performance.

Table 5.1: KAN models used for performance evaluations. 784 is the number of pixels in an MNIST image, 10 is the number of classes. G denotes the grid size and k the spline order.

#	Shape	G/k	Param. count (K)	Accuracy (%)		
				FP32	INT8	HW
1	784, 24, 10	2/1	57	96.63	96.60	96.63
2	784, 24, 10	4/1	95	96.66	96.44	96.36
3	784, 64, 10	2/1	152	97.34	97.30	97.24
4	784, 24, 24, 10	2/1	58	95.72	95.62	95.64
5	784, 24, 24, 10	6/1	137	95.03	94.96	94.96

5.1 Validation

The hardware architecture and optimization techniques developed throughout this work were implemented on a Xilinx Artix-7 FPGA, deployed using the Digilent Nexys-4 development board as the physical test platform. To validate the functional correctness of the accelerator, a SystemVerilog testbench was constructed. This testbench utilized an AHB interface to communicate with the accelerator and replicated a realistic inference flow. It initialized the system with quantized model weights, configuration parameters, and input images from the MNIST dataset, stored in .txt files. The testbench executed inference on the full MNIST test set consisting of 10,000 images and compared the predicted labels from the accelerator against the ground-truth labels to verify functional correctness.

For verification across the design flow, the accelerator was simulated at behavioral, post-synthesis, and post-implementation levels. This confirmed that synthesis and place-and-route steps preserved the intended Register-Transfer Level (RTL) behavior, ensuring the final bitstream performed inference as expected.

Inference was performed in batches of eight MNIST images, covering the entire test set of 10,000 images. However, the choice of batch size has no impact on the results, as the accelerator processes one image at a time and does not support parallel batch execution. Consequently, running the test set in a single batch would yield identical outcomes. For each batch, the loading process via the AHB interface consistently required 1631 Clock Cycles (CCs). This value remained constant across all models, as it depends solely on the fixed size of the MNIST image data and not on the model architecture. The resulting accuracies are presented in the Table 5.1.

The INT8 accuracies reported in Table 5.1 do not always match those observed in the hardware results shown in Table 5.1. This discrepancy arises from the difference in the LUT architectures: the Python simulations assumed a full-LUT configuration, while the hardware implementation used an HB-LUT design. In some instances, this led to slight accuracy improvements (e.g., Models #1 and #4), in others the accuracy remained unchanged (Model #5), and in a few cases it decreased slightly (Models #2 and #3). Nonetheless, these variations are minor and can be considered negligible when weighed against the hardware efficiency benefits of using HB-LUT design.

5.2 Timing and Performance

The accelerator was integrated into a system built around the 32-bit AMD MicroBlaze RISC soft-core processor. To simplify deployment, the MNIST input activations and network weights were precomputed and loaded into the KAN accelerator using .coe files supported by Vivado. During inference, the MicroBlaze executed C code that continuously polled the accelerator’s status register until a READY state, indicating completion. This procedure is followed in addition to the validation discussed above to make sure that the system design is able to run

on the actual silicon.

The complete system was synthesized with a target clock frequency of 62.5MHz on the FPGA, achieving a worst-case setup slack of 0.820ns. This critical path began in the PE, passed through the RQ, and ended in the A-RAM - hence, triggered when writing results from the MVME to the A-RAM. Another near-critical path with 0.863ns slack extended from the C-RAM to the PE registers. These timing paths highlight areas for future architectural optimization using pipelining techniques - such as inserting additional registers - at the cost of increased latency.

In addition to clock frequency, two critical performance metrics for digital hardware are throughput and latency. In the context of ML accelerator design, throughput refers to the number of inferences the system can complete per second. It is defined as follows [19]:

$$\text{throughput} = \frac{\text{inferences}}{\text{second}} = \frac{\text{operations}}{\text{second}} \times \frac{1}{\frac{\text{operations}}{\text{inference}}} \quad (5.1)$$

For a KAN model, the number of operations required per inference is given by:

$$\frac{\text{operations}}{\text{inference}} = 2 \cdot \sum_{i=0}^{L-1} n_i \cdot n_{i+1} \quad (5.2)$$

where L denotes the total number of layers, and n_i represents the number of nodes in the i^{th} layer. In this work, end-to-end throughput is reported - this includes not only the computation time of the inference, but also the overhead associated with loading a batch of MNIST images.

Inference time was measured in the SystemVerilog simulation as the duration between initiating the accelerator after loading a batch of images and the moment the accelerator asserted the READY signal. As discussed earlier, loading a batch of eight images into the accelerator took 1631 CCs, equivalent to 26,096ns.

Using these measurements, the end-to-end throughput is calculated as:

$$\text{throughput} = \frac{8}{\text{inference time} + \text{data load time}} \quad (5.3)$$

The number of operations per second is then obtained by multiplying the throughput in Eq. (5.3) with the number of operations per inference given in Eq. (5.2), as defined in Eq. (5.1).

PE utilization, which reflects how efficiently the architecture leverages available parallelism, is computed as [19]:

$$\text{PE utilization} = \frac{\text{actual GOPS}}{\text{ideal GOPS when util. is 100\%}} = \frac{\text{actual GOPS}}{(2 \times 62.5\text{MHz}) \times 4} \quad (5.4)$$

Finally, latency is defined as the time between the system receiving a single input image and producing the corresponding output. It is measured in seconds and represents the system's response time. It was calculated from the aforementioned measurements by:

$$\text{latency} = \frac{1}{\text{throughput}} \quad (5.5)$$

Table 5.2: Performance metrics for the reference KAN models.

#	Ops/Inf	GOPS	Inference time (μ s)	Throughput (inf/s)	Latency (μ s)
1	38112	0.47	617	12,430	80.45
2	38112	0.47	617	12,430	80.45
3	101632	0.49	1,639	4,802	208
4	39264	0.47	637	12,056	82.94
5	39264	0.47	637	12,056	82.94

Table 5.2 summarizes these key performance metrics for the five reference models executed on the designed accelerator. Notably, models with identical architectures but varying grid sizes exhibited the same inference time. This demonstrates that the implemented design optimizations successfully decoupled the computational complexity of inference from the grid size. For instance, although Model #2 has more parameters than Model #1, both achieved the same inference time.

a PE utilization is approximately 96%. This figure includes not only the time spent actively computing but also the idle clock cycles where PEs were stalled while waiting for MNIST data to be loaded into the A-RAM.

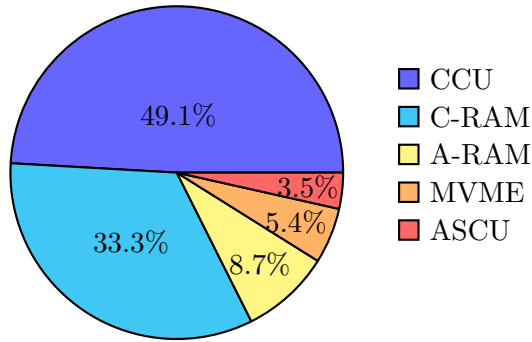
5.3 FPGA Utilization

Table 5.3 summarizes the resource utilization of the designed KAN accelerator on the target FPGA device. The most heavily used component type is the Block RAM (BRAM), where approximately 46% of the available memory blocks are consumed. This high BRAM usage is primarily due to supporting a wide range of KAN model weights locally on-chip, eliminating the need for frequent external memory access or weight reloading. The relatively high utilization of BRAMs suggests that memory footprint can be a potential bottleneck for scalability, particularly when targeting devices with fewer memory resources.

In contrast to the high BRAM usage, the utilization of logic elements - specifically LUTs and Flip-Flops (FFs) - remains relatively low, with only 4.5% of LUTs and 2.81% of FFs used. Similarly, the usage of DSP slices is modest at 6.67%. These arithmetic units are primarily used for the multiply-accumulate operations involved in PEs. This suggests that the logic complexity of the accelerator is modest compared to the available FPGA resources, leaving substantial headroom for future extensions assuming enough memory throughput. For example, the design could be expanded by adding more PEs or by incorporating support for evaluating higher-order B-splines, all while staying within the available resource constraints.

Table 5.3: Utilized FPGA resources divided by type.

Site Type	Used	Available	Util (%)
LUTs	2851	63400	4.50
FFs	3561	126800	2.81
BRAMs	62	135	45.93
DSPs	16	240	6.67

**Figure 5.1:** Breakdown of dynamic power consumption by accelerator submodules.

5.4 Power

A breakdown of the dynamic power consumption is shown in Figure 5.1. The CCU is the most power-consuming submodule, accounting for 49.1% of the dynamic power budget. This is expected, as the CCU is responsible for controlling access to both the A-RAM and C-RAM, and transferring data to the MVME for computation.

The C-RAM follows with 33.3% of dynamic power, which is reasonable given that coefficient values are accessed frequently during inference. In contrast, the smaller and less frequently accessed A-RAM consumes only 8.7%. This suggests an opportunity for future power optimizations, such as clock gating the C-RAM during input loading phases or other similar techniques.

The MVME and ASCU together consume less than 10% of the dynamic power budget. The low power usage of the MVME indicates that the arithmetic computations are already power-efficient, and the design could potentially scale to support additional parallelism, provided memory and control logic power remain manageable.

5.5 Summary

Table 5.4 lists the main hardware specifications of the implemented accelerator.

Table 5.4: Accelerator specifications.

Specification	Value
On-chip memory size (kB)	272
Number of PEs	4
PE utilization (%)	96
Clock frequency (MHz)	62.5
Data-width	8-bit
Dataset	MNIST
Performance (GOPS)	0.47
Supported models (flexibility)	Number of layers: ≤ 8 Layer width: ≤ 4096 G : ≤ 16 k : 1 Batch size: ≤ 10

The design includes 272kB of on-chip memory and features 4 PEs. According to profiling results, the PE utilization was measured to be **96%**, suggesting that the control and dataflow logic kept the computational units active for most of the inference cycle.

The clock frequency was set to 62.5MHz, and computations were performed using 8-bit integer data. The accelerator was evaluated on the MNIST dataset, achieving a throughput of 0.47GOPS and a power efficiency of 3.11GOPS/W based on Vivado’s power estimation.

The hardware supports a range of model configurations, including up to eight layers with a layer width of 4096. It supports up to $G = 16$ grid points, and only $k = 1$ spline order, and a maximum batch size of 16. These parameters reflect the operational limits and model support under the current implementation.

Overall, the section presents a quantitative overview of the design’s current capabilities and tested configurations.

Conclusion

This work presented one of the first comprehensive hardware implementations of KANs, introducing a series of both algorithmic and architectural optimizations. At the algorithmic level, a quantization technique was developed to reduce the data-width of network parameters and B-spline functions while preserving accuracy, enabling the use of lightweight MACs and reducing memory footprint. A simplified LUT (HB-LUT) design was introduced to evaluate B-spline basis functions efficiently, eliminating redundant entries and minimizing memory usage. An optimized inference algorithm was developed to bypass ineffectual coefficients - those associated with basis functions that evaluate to zero - thereby reducing computational complexity of the forward pass.

On the hardware side, the memory layout of coefficients was structured to minimize addressing overhead and to support selective coefficient access. A specialized PE architecture tailored for first-order B-splines was also developed, supporting the inference algorithm further. Finally, a dedicated module, the comparator chain, was implemented to identify the active basis functions for each input, enabling the system to determine which coefficients can be bypassed during the inference.

These optimizations endowed KANs with a unique ability to scale their parameter count, primarily through grid extension, without incurring additional computational overhead. Despite this advantage, empirical results revealed that simply increasing the number of parameters does not guarantee improved accuracy. Specifically, experiments on the MNIST dataset demonstrated that raising the grid resolution beyond two or employing spline orders greater than one results in only marginal gains in classification accuracy. These observations suggest that the expressive potential of KANs are not fully utilized in image classification tasks. Nevertheless, in problem domains where both increased non-linearity and higher grid sizes are beneficial, these optimizations could provide KANs with a distinct advantage over traditional neural networks. Reinforcement learning applications in robotic control, for instance, may offer a more suitable benchmark, as such tasks typically demand richer function approximation and could benefit from KANs' inherent ability to learn interpretable, mathematically grounded structures.

Future Work

As with all scientific work, there remains significant room for improvement and future exploration. At a higher level:

1. **Training Frameworks.** The training libraries used in this work - namely PyKAN and Efficient-KAN - proved to be a major bottleneck, consuming a substantial portion of development time. These frameworks, while promising, lack the robustness and scalability required for training larger KAN models such as those targeting MNIST. Key features like grid extension and symbolification either failed to function as expected or produced suboptimal results at this scale. Moreover, the dynamic grid adjustment mechanism, which updates grid based on the input range of each AF, often led to numerical instability during training. Developing a more specialized and stable training library tailored to support larger KAN models and better aligned with hardware constraints would significantly enhance both model performance and hardware efficiency, while also unlocking new opportunities for architectural innovation. Another interesting direction for future work would be to enable layer-specific grid sizes, allowing each layer to have its own resolution. This would increase the internal degrees of freedom in KANs and potentially enhance their representational capacity.
2. **Quantization.** While the current quantization approach delivered satisfactory results, there is potential to explore more aggressive schemes, such as reducing parameter and activation widths to four bits or lower. Additionally, the implemented strategy assumes a static grid structure in KANs. Although this was sufficient for the present work, developing a quantization method tailored for dynamic grids would be a valuable and potentially standalone research direction.
3. **Inference Algorithm.** The algorithm developed in this work supports only first-order B-spline evaluations, a deliberate choice given their sufficiency for image classification tasks. However, extending the algorithm to support higher-order B-splines - particularly cubic B-splines, which are the most commonly used in the original KAN research - would be a valuable enhancement. Such an extension would broaden the applicability of the

accelerator and align it more closely with mainstream KAN models.

At the hardware level:

1. **Clock Speed.** The current accelerator operates at 62.5MHz on the target FPGA, which is sufficient for proof-of-concept design. However, future designs aiming for higher performance could benefit significantly from a more deeply pipelined architecture. By identifying and restructuring the critical paths - particularly in the coefficient fetch, base function evaluation, and MAC units - the design could achieve higher clock frequencies. Introducing register stages between key processing blocks, decoupling memory accesses from computation, and optimizing combinational logic would all contribute to a faster, more scalable accelerator suitable for deployment in latency-sensitive or high-throughput environments.
2. **Memories.** As discussed earlier, the C-RAM was implemented using two single-port RAM blocks to emulate dual-port behavior, with the goal of avoiding dual-port RAMs. In retrospect, this decision was uninformed - Xilinx FPGA block RAMs are inherently dual-port, meaning the current design inadvertently used two dual-port RAMs to avoid using just one. If the accelerator were intended strictly for FPGA deployment, this inefficiency could be eliminated by replacing the banked RAM structure with a native dual-port RAM implementation, reducing resource usage and simplifying control logic. However, since the FPGA was primarily used as a prototyping platform to validate the accelerator's functionality, this was not a critical issue. Importantly, if the design were to be fabricated as an ASIC, the current memory architecture - using separate banks - would still be beneficial.
3. **Re-quantization Units.** Another area for improvement lies in reducing the number of RQs within the accelerator. Each RQ includes a larger multiplier than the 8-bit ones used in the PEs, making them more area-intensive. Currently, each PE is paired with its own dedicated RQ, but this could be optimized. A more efficient approach would involve sharing a single RQ module across all PEs, sequentially rescaling their outputs over four clock cycles. This would significantly reduce hardware overhead. Interestingly, this optimization aligns well with the A-RAM's structure: each row contains four bytes. While the input side processes one row over four cycles, the write-back logic could simultaneously handle RQ operations, accumulating and writing back the rescaled results. This pipelined approach would maintain throughput while reducing area usage.

Looking ahead, the integration of novel neural architectures like KANs with efficient hardware accelerators represents a promising direction for sustainable and high-performance machine learning. As demands on edge and embedded systems grow, approaches that rethink fundamental components - such as activation functions and dataflow architectures - will be key to overcoming current limitations. This thesis contributes a meaningful step in that journey, and it is hoped that the insights and designs presented here will inspire continued innovation at the

intersection of algorithm development and hardware implementation, ultimately enabling smarter, faster, and more energy-conscious intelligent systems.

References

- [1] O. Y. Al-Jarrah, P. D. Yoo, S. Muhaidat, G. K. Karagiannidis, and K. Taha, “Efficient Machine Learning for Big Data: A Review,” 2015.
- [2] V. Sze, Y.-H. Chen, J. Emer, A. Suleiman, and Z. Zhang, “Hardware for machine learning: Challenges and opportunities,” in *2017 IEEE Custom Integrated Circuits Conference (CICC)*, IEEE, Apr. 2017.
- [3] S. Mehta, “How Much Energy Do LLMs Consume? Unveiling the Power Behind AI,” July 2024. Accessed: 2025-05-20.
- [4] G. Bai, Z. Chai, C. Ling, S. Wang, J. Lu, N. Zhang, T. Shi, Z. Yu, M. Zhu, Y. Zhang, X. Song, C. Yang, Y. Cheng, and L. Zhao, “Beyond Efficiency: A Systematic Survey of Resource-Efficient Large Language Models,” 2024.
- [5] K. Hornik, M. Stinchcombe, and H. White, “Multilayer feedforward networks are universal approximators,” *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989.
- [6] U. Sharma and J. Kaplan, “A Neural Scaling Law from the Dimension of the Data Manifold,” Apr. 2020. arXiv:2004.10802 [cs].
- [7] J. Jumper, R. Evans, A. Pritzel, *et al.*, “Highly accurate protein structure prediction with AlphaFold,” *Nature*, vol. 596, pp. 583–589, 2021.
- [8] A. N. Kolmogorov, “On the representation of continuous functions of many variables by superpositions of continuous functions of one variable and addition,” *Doklady Akademii Nauk SSSR*, vol. 114, pp. 953–956, 1957. English translation available in *AMS Translations - Series 2*, Vol. 28, 1963, pp. 55–59.
- [9] V. I. Arnold, “On functions of three variables,” *Matematicheskii Sbornik*, vol. 30, no. 2, pp. 71–74, 1957.
- [10] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark, “KAN: Kolmogorov-Arnold Networks,” Feb. 2025. arXiv:2404.19756.
- [11] R. Bellman, *Dynamic Programming*. Princeton University Press, 1957.

- [12] E. J. Michaud, Z. Liu, and M. Tegmark, “Precision Machine Learning,” *Entropy*, vol. 25, p. 175, Jan. 2023. arXiv:2210.13447 [cs].
- [13] V. D. Tran, T. X. H. Le, T. D. Tran, H. L. Pham, V. T. D. Le, T. H. Vu, V. T. Nguyen, and Y. Nakashima, “Exploring the Limitations of Kolmogorov-Arnold Networks in Classification: Insights to Software Training and Hardware Implementation,” 2024.
- [14] W.-H. Huang, J. Jia, Y. Kong, F. Waqar, T.-H. Wen, M.-F. Chang, and S. Yu, “Hardware Acceleration of Kolmogorov-Arnold Network (KAN) for Lightweight Edge Inference,” 2024.
- [15] F. Holmér, “The Continuity of Splines.” <https://www.youtube.com/watch?v=jvPPXbo87ds>, 2022. Accessed: 2025-05-21.
- [16] C. de Boor, “Subroutine package for calculating with B-splines,” Technical Report LA-4728-MS, Los Alamos Scientific Laboratory, Los Alamos, NM, 1971.
- [17] M. G. Cox, “The numerical evaluation of B-splines,” *Journal of the Institute of Mathematics and its Applications*, vol. 10, no. 2, pp. 134–149, 1972.
- [18] Z. Liu, “PyKAN: A Python Library for Kolmogorov–Arnold Networks.” <https://github.com/KindXiaoming/pykan>, 2024. Accessed: 2024-12-12.
- [19] V. Sze, Y. Cheng, T.-J. Yan, and J. S. Emer, *Efficient processing of deep neural networks*. No. 50 in Synthesis lectures on computer architecture, Cham: Springer, reprint of original edition morgan & claypool 2020 ed., 2022.

Extra material

Listing A.1: C code example demonstrating communication with the KAN accelerator. Assumes coefficients and input images are preloaded.

```

1 #include <stdio.h>
2 #include "platform.h"
3 #include "xil_printf.h"
4 #include "xil_io.h"
5
6 int main()
7 {
8     init_platform();
9
10    // set up the configuration registers
11    Xil_Out32(0x1A250000, 0x00000042);
12    Xil_Out32(0x1A250080, 0x0C400612);
13    Xil_Out32(0x1A250084, 0x00600292);
14    // ... several lines skipped
15    Xil_Out32(0x1A250340, 0x7F000000);
16
17    // set the start bit
18    Xil_Out32(0x1A260000, 1);
19
20    // poll the status bit
21    while (Xil_In32(0x1A260004) != 1);
22
23    // print results
24    for (int i = 0; i < 4*3*8; i+=4) {
25        xil_printf("%lx\r\n", Xil_In32(0x1A240000+i));
26    }
27
28    cleanup_platform();
29    return 0;
30 }
```

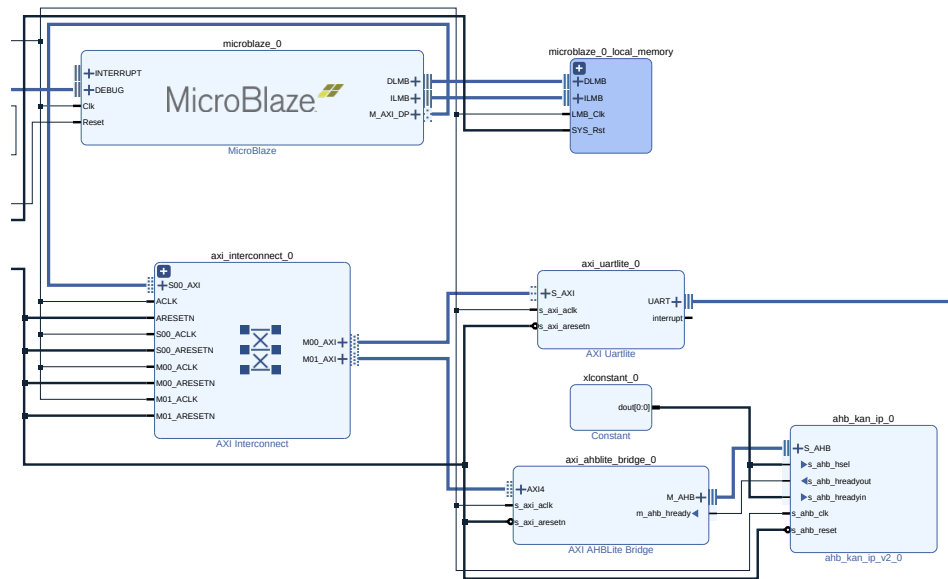


Figure A.1: AHB KAN peripheral in a MicroBlaze system.

Listing A.2: Comparator chain implementation in SystemVerilog.

```

1 integer i;
2 logic found;
3 always_comb begin : comparator_chain
4     found = 1'b0;
5     n_input_grid_index=0;
6
7     for (i=0; i<16; i=i+1) begin
8         if (($signed(r_conf.grid[r_cnt.layer][i]) >
9             n_current_input) &&
10            (!found) &&
11            (i <= s_max_knot_index))
12             begin
13                 n_input_grid_index = i;
14                 found = 1;
15             end
16     end

```